



PatchTrack: A comprehensive analysis of ChatGPT's influence on pull request outcomes

Daniel Ogenrwot¹ · John Businge¹

Received: 18 August 2025 / Accepted: 21 April 2026 / Published online: 7 May 2026

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract

The rapid adoption of large language models (LLMs) like ChatGPT has introduced new dynamics in software development, particularly within pull request workflows. While prior research has examined the quality of AI-generated code, less is known about how developers evaluate, adapt, and integrate these suggestions in real-world collaboration. We analyze 338 pull requests from 255 GitHub repositories containing self-admitted ChatGPT usage, comprising 645 AI-generated snippets and 3,486 developer-authored patches. To support this analysis at scale, we use PatchTrack, an automated classifier that identifies whether AI-generated patches were applied, partially reused, or not integrated. Our findings reveal that full adoption of ChatGPT-generated code is uncommon: the median integration rate is 25%. Qualitative analysis of 89 pull requests with integrated patches reveals recurring patterns of *structural integration*, *selective extraction*, and *iterative refinement*, indicating that developers typically treat AI output as a starting point rather than a final implementation. Even when code is not directly adopted, ChatGPT influences workflows through *conceptual guidance*, *documentation*, and *debugging strategies*. Integration decisions reflect *contextual fit*, *integration effort*, *maintainer trust*, and established pull request review norms rather than serving as direct indicators of code correctness. Overall, this study provides empirical insight into AI-mediated decision-making in collaborative software development, showing that the influence of generative AI extends beyond patch generation to how developers reason about, adapt, and negotiate code during review within pull request workflows. These findings inform the design of AI-assisted tools and support more transparent and effective use of LLMs in practice.

Keywords ChatGPT · Pull requests · Code adoption · AI-assisted development · Self-admitted AI usage · Developer-in-the-loop · AI-assisted code review

Communicated by: Hideaki Hata.

Extended author information available on the last page of the article

1 Introduction

AI-assisted workflows are increasingly shaping software development, with tools such as ChatGPT and GitHub Copilot that promise improved automation, productivity, and collaboration (Menzies and Pecheur 2020; Russo 2024; Huang et al. 2024; Ebert and Louridas 2023; Sauvola et al. 2024). Prior work has examined their utility in code generation, debugging, and documentation (Ozpolat et al. 2023; Peng et al. 2023; Wermelinger 2023; Jin et al. 2024), but most studies have focused on controlled settings or isolated tasks. Their impact on collaborative workflows, particularly pull request (PR) decision-making, remains poorly understood (Grewal et al. 2024; Siddiq et al. 2024). Understanding how developers integrate, modify, or reject AI-generated code in real-world PRs is essential to improving AI-assisted collaboration.

Consider a common scenario in open-source development: a contributor submits a pull request and notes that part of the patch was suggested by ChatGPT. During review, maintainers must decide whether the suggested code should be integrated as-is, adapted, or rejected. Objections often concern contextual fit, architectural alignment, or interaction with surrounding code rather than the underlying idea. As a result, contributors may revise the patch, extract only fragments of the suggestion, or abandon it entirely. Such outcomes are typically invisible in analyses that rely solely on merge status or coarse-grained pull request metadata. This study is motivated by the need to empirically characterize these decision-making processes at scale and to examine how AI-generated suggestions influence pull request outcomes even when the code itself is modified or rejected.

To understand why these decision processes matter, it is important to situate them within the broader role of pull requests in modern software engineering. Pull requests enable peer review, discussion, and integration (Rigby and Bird 2013; Bacchelli and Bird 2013; Storey et al. 2017). Traditional studies have shown that PR outcomes depend on factors such as patch size, contributor history, and code quality (Gousios et al. 2014; Tsay et al. 2014; Gousios et al. 2016; Soares et al. 2015; Zhu et al. 2016). Recently, tools like GitHub Copilot have entered the PR workflow (Xiao et al. 2024), offering autogenerated summaries and suggestions to speed up reviews. These advances echo the long-standing vision of the *Programmer's Apprentice* (Rich and Waters 1990), in which AI augments rather than replaces human reasoning. Yet, the effectiveness of such tools in shaping real pull request decisions remains poorly understood.

Although prior research has explored predictive models and recommendation systems to support pull request decision-making (Zhao et al. 2019; Azeem et al. 2020; Dey and Mockus 2020), these studies typically rely on social and technical metadata such as contributor history or patch size rather than on the content of patches themselves. In parallel, research in AI-assisted software engineering has focused primarily on improving code quality, debugging, documentation, and developer productivity through generative models (Hassan et al. 2024; White et al. 2023; Luo et al. 2024; Howard and Ruder 2018; Jiang et al. 2023; Guo et al. 2024; Deng et al. 2024; Hou et al. 2023; Ju et al. 2023; Siddiq et al. 2023). While these studies demonstrate technical and productivity benefits, often in controlled or task-specific settings, they do not fully examine how AI-generated outputs are judged, adapted, or discarded in collaborative workflows such as pull requests.

Even studies of ChatGPT-generated code that report integration rates and patch retention (Grewal et al. 2024; Siddiq et al. 2024) do not fully capture the nuanced decisions

developers make during pull request discussions. To move beyond binary merge outcomes, we therefore examine whether developers apply ChatGPT-suggested code directly, modify it substantially, act only on conceptual guidance, or ultimately reject it, providing a more fine-grained view of AI-mediated decision-making in pull request workflows.

Meanwhile, developers are increasingly incorporating ChatGPT into daily development (Tufano et al. 2024a), yet recent evaluations of automated code review tools show that such systems often overlook human decision-making context, leading to challenges in accuracy, trust, and integration (Tufano et al. 2024b). Hallucinated or incorrect code further undermines adoption, as developers must verify or revise suggestions (Tanzil et al. 2024). Multi-hunk patches pose additional challenges for large language models due to limitations in semantic coordination and global reasoning (Nashid et al. 2025). While these technical and socio-technical barriers are increasingly recognized, few studies have examined them empirically in the context of real-world pull request outcomes.

This study addresses that gap by analyzing 338 pull request conversations from open-source projects involving self-admitted ChatGPT usage (SACU). We introduce and apply *PatchTrack* as an analytical instrument for classifying patch outcomes and usage patterns under AI assistance, enabling large-scale empirical observation of patch-level integration outcomes rather than providing a developer-facing or prescriptive tool. Originally presented as a poster at ASE 2024, we significantly extend *PatchTrack*'s capabilities by improving classification precision, validating its outputs, and applying it to a larger, SACU-enhanced version of the DevGPT dataset (Xiao et al. 2023). This enables a scalable and systematic analysis of how ChatGPT-generated suggestions are integrated, adapted, or rejected in collaborative workflows.

Unlike prior work that focuses primarily on integration rates or isolated productivity metrics (Jiang et al. 2023; Guo et al. 2024; Deng et al. 2024; Hou et al. 2023; Ju et al. 2023; Siddiq et al. 2023; Li et al. 2024), we adopt a behavioral lens that examines how developers interpret, adapt, or disregard ChatGPT's outputs within real-world team workflows. Our findings reveal that developers frequently use ChatGPT for purposes beyond code generation, such as conceptual guidance, naming, documentation refinement, and debugging support. We observe a spectrum of use, from direct integration to complete rejection, shaped by contextual judgment, trust, and project norms. This perspective contributes a more grounded understanding of AI-assisted collaboration, informing both tool design and responsible adoption.

The paper makes the following contributions:

- We introduce and apply *PatchTrack* as an analytical instrument to empirically study how developers integrate, adapt, or reject ChatGPT-generated code in real pull requests.
- We provide a mixed-methods analysis of applied patches, identifying recurring patterns of *structural integration*, *selective extraction*, and *iterative refinement*, showing that developers treat AI-generated code as a starting point rather than a final solution.
- We show that ChatGPT influences pull request outcomes even when no code is adopted, through conceptual guidance, documentation refinement, and debugging support.
- We identify cases where valid AI-generated patches are discarded due to architectural misfit, contributor role, or workflow norms, highlighting socio-technical barriers to adoption.
- We release a public replication package (Ogenrwot and Businge 2025) to support repro-

ducibility and future research on AI-assisted collaboration.

2 Terminology, Problem, and Concrete Examples

In this section, we define key terminology used in this study, outline the classification of ChatGPT-assisted developer interactions, and present a concrete example to illustrate these concepts.

2.1 Terminology

- **Code snippet:** A small, standalone block of code intended for reuse, demonstrating a specific task or programming concept. In this study, “code snippet” refers exclusively to discrete pieces of code that ChatGPT suggests, not including code examples embedded within descriptive text or theoretical guidance.
- **Diff:** The difference between two versions of files, highlighting the changes between an original and a modified file.
- **Patch:** A file containing differences between two versions of one or more files. A patch is a specific type of diff that outlines the changes from one state to another.
- **Hunk:** A contiguous block of changes within a diff or patch, indicating differing lines between two versions of a file (GitHub: Online Appendix 2025). Hunks are identified by headers in the format @@ -s, n +s, n @@, which mark the start and the number of lines affected in the patch.
- **Self-admitted ChatGPT usage (SACU):** Instances where developers explicitly acknowledge, within a pull request discussion or issue thread, that they used ChatGPT to generate, modify, or assist in writing code. This admission is typically evident in pull request comments, commit messages, discussion threads, or issue reports where developers refer to AI-generated suggestions.
- **PatchTrack:** A classification tool that determines whether ChatGPT-generated patches were applied, not applied, or not suggested during developers’ integration with ChatGPT.

2.2 Classifying Developer Interaction with ChatGPT

A typical GitHub pull request workflow augmented with ChatGPT interactions can originate from the pull request author or a reviewer. Consider a scenario in which a developer, intending to fix a bug, add a feature, or improve an existing one, initiates a change to a file named `foo`. Following a pull-based development workflow (Gousios et al. 2014), the developer creates a new branch, commits changes, and eventually merges the pull request into the main repository of the project.

During development, the developer might encounter challenges or seek optimal solutions, prompting them to consult ChatGPT. The responses provided by ChatGPT may be textual explanations or code snippets. In this study, we focus on self-admitted ChatGPT usage (SACU), where developers explicitly acknowledge using ChatGPT within pull

request discussions. Based on the self-admitted ChatGPT usage interactions, we classify ChatGPT-assisted pull requests into three possible scenarios:

1. When the response includes a code snippet, the developer may apply this directly to `FOO` as a patch, either as provided or after modifications. This scenario is labeled as “Patch Applied (PA)”.
2. If the code snippets are not applied, or modified beyond recognition (resulting in a semantic clone), then `FOO` may remain unchanged or include changes influenced by the discussed concepts. This scenario is defined as “Patch Not Applied (PN)”.
3. If the interaction with ChatGPT involves only textual advice without code snippets, similar to the previous scenario, `FOO` may remain unchanged or may be modified based on the guidance received. This scenario is referred to as “No Existing Patch (NE)”.

This categorization underpins our analysis by structuring the diverse ways developers interact with ChatGPT in pull requests. By distinguishing between directly applied patches (PA), substantially modified or conceptually reused suggestions (PN), conceptual-only interactions without code (NE), and rejected contributions in closed pull requests (CL), we can (i) trace ChatGPT’s concrete influence on code changes, (ii) identify barriers to adoption, trust, and contextual fit, (iii) examine decision-making dynamics in AI-assisted workflows, and (iv) analyze failure cases where AI involvement did not lead to integration. The inclusion of CL cases is particularly valuable for understanding how project constraints, scope misalignment, or collaboration breakdowns affect AI-supported contributions. This classification also informs targeted recommendations: improving the usability of the patch (PA), aligning suggestions with developer expectations (PN), improving conceptual guidance (NE) and mitigating risks in unmerged PRs (CL). Together, these categories link observed outcomes to distinct usage patterns, supporting evidence-based insights into how generative AI can enhance the efficiency and reliability of collaborative code review.

Our classification relies on the self-admitted ChatGPT usage, meaning that only cases where developers explicitly state that ChatGPT was used are included in the dataset. This ensures transparency in identifying AI-assisted development while acknowledging that instances of non-disclosed ChatGPT usage remain outside the scope of this study.

2.3 Concrete Example

To ground the terminology used in this study, we present a real-world example of a “PA” scenario. Concrete examples of “PN” and “NE” cases are discussed later in Section 4.3. In this example, a developer consulted ChatGPT for assistance with a continuous integration issue, and ChatGPT provided a code snippet to address the problem. Figure 1 shows a portion of this interaction. The full dialogue is available online (GitHub pull request [2023a](#)) and is referenced in pull request #2405 (GitHub pull request [2023b](#)) from the `faker-js/faker` repository.

Analysis of the corresponding diff (Commit: 62eb120, Hunk: @@ -0,0 +1,31 @@) shows that the developer integrated the ChatGPT-recommended code verbatim before merging the pull request on September 20, 2023. This case exemplifies direct patch application and is categorized as PA. This example serves to concretely ground the classification used throughout the study and motivates the research questions that examine how frequently

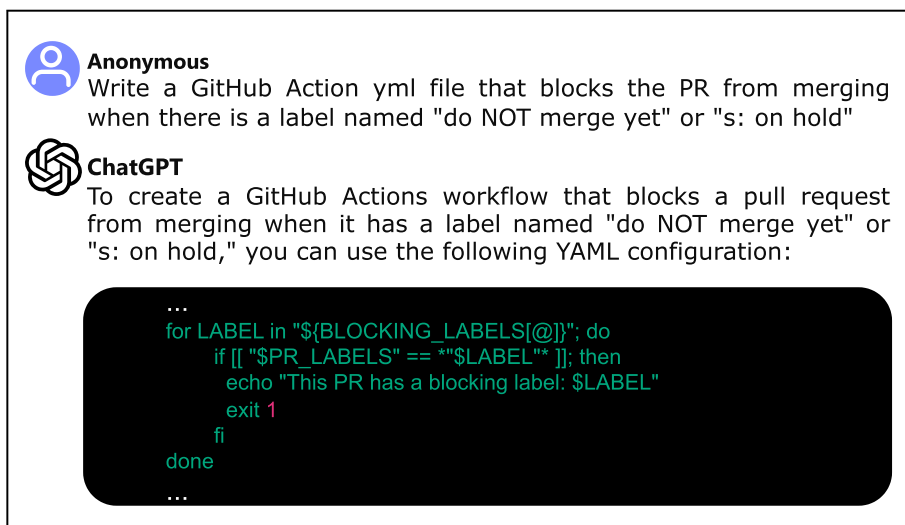


Fig. 1 Conversation between developer and ChatGPT

such patterns occur and how developers integrate, adapt, or discard AI-generated suggestions in pull request workflows.

Beyond the scenarios where the pull request is merged, we also analyze cases where a pull request is closed without merging, despite developers sharing ChatGPT conversation links within the pull request discussions. In these instances, the developer creates a new branch, modifies files, and submits a pull request following the standard workflow. ChatGPT may have assisted with code suggestions, debugging, or conceptual guidance, yet the pull request is ultimately closed due to factors such as scope misalignment, quality concerns, or project constraints. We refer to this scenario as the Closed Case (CL). This study investigates the reasons behind CL, examining how ChatGPT's involvement influenced closure decisions and whether AI-assisted contributions shaped the final outcome.

3 Study Design

This section presents the research method from data collection, tool design, and quantitative and qualitative analysis. The overarching study design is illustrated in Fig. 2.

3.1 Research Questions

- **RQ1:** *How are ChatGPT-generated patches used and integrated in pull requests under self-admitted ChatGPT usage, as observed through patch-level classification?* This RQ investigates both the distribution and nature of ChatGPT-generated patch integration in merged pull requests. We use PatchTrack to classify pull requests into three categories: (i) patches that were applied (PA), (ii) patches that were suggested but not applied (PN), and (iii) cases where ChatGPT did not generate patches (NE). In addition to this

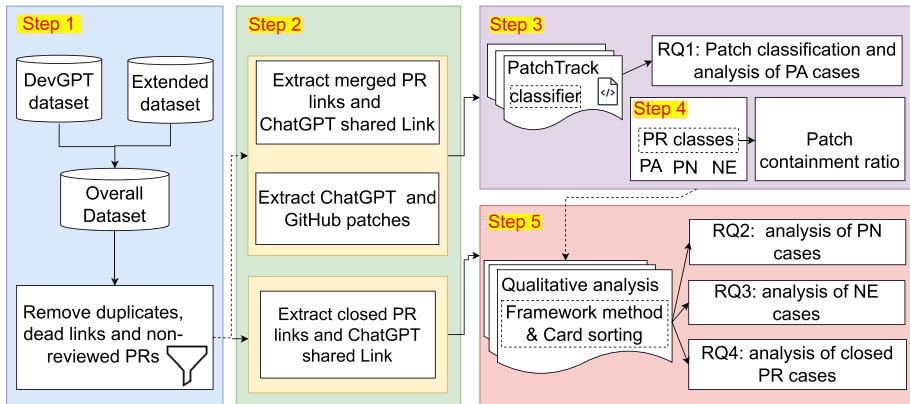


Fig. 2 Overview of the study method

quantitative view, we conduct a thematic analysis of PA cases to understand how developers adopt, reshape, or refine ChatGPT's contributions. Together, these perspectives provide a comprehensive view of AI-generated patch adoption and its role in collaborative development.

- **RQ2:** *Why are ChatGPT-suggested patches in conversations not directly integrated (PN) into the pull request, and how do developers use ChatGPT's suggestions in their workflow under the self-admitted ChatGPT usage?* Understanding why developers choose not to integrate ChatGPT-generated patches can provide insights into the challenges and constraints that influence AI-assisted development. Furthermore, examining how developers engage with ChatGPT suggestions can shed light on the broader role of AI in software engineering workflows.
- **RQ3:** *Under what circumstances does ChatGPT not generate patches in developer interactions (NE), and how do developers use its responses under the self-admitted ChatGPT usage?* This RQ examines cases where ChatGPT does not suggest direct code patches, exploring the reasons behind such interactions. In addition, it investigates how developers incorporate the responses of ChatGPT - whether for conceptual understanding or other non-patch-related tasks - into their development workflows.
- **RQ4:** *How does ChatGPT influence closed pull requests, and what factors contribute to these outcomes under the self-admitted ChatGPT usage?* This RQ examines the relationship between ChatGPT-assisted contributions and closed pull requests. It aims to explore the factors that lead to pull request closures in cases where ChatGPT-generated content is referenced, providing insights into the broader role of AI-assisted development in collaborative software engineering.

3.1.1 Scope and Unmodeled Factors

Our study does not control for prompt formulation or developer expertise, though both can influence the quality and integration of AI-generated code. Prior work shows that incomplete or ambiguous prompts can yield suboptimal suggestions (Ehsani et al. 2025), and that developers often adapt or reject AI-generated code based on perceived quality (Grewal et al. 2024; Siddiq et al. 2024). We instead focus on self-admitted ChatGPT usage in real-world

pull requests, enabling grounded analysis of integration behavior without assumptions about prompt content. These unmodeled factors affect all classes in our study (PA, PN, NE, CL) and are unlikely to bias classification or thematic coding disproportionately. We highlight prompt quality and user expertise as valuable future directions for modeling variation and supporting cross-tool comparisons in AI-assisted development.

3.1.2 Rationale for Categorization

We classify ChatGPT-assisted pull requests into four categories: Patch Applied (PA), Patch Not Applied (PN), No Patch Generated (NE), and Closed (CL). This categorization is central to our study as it enables a structured understanding of how developers engage with AI-generated suggestions. PA cases allow us to trace direct integration of ChatGPT-generated code. PN cases highlight friction points such as lack of trust, scope misalignment, or manual refinement that prevent full adoption. NE cases reveal how developers use ChatGPT for conceptual understanding or planning, even when no code is generated. Finally, CL cases expose collaborative or technical breakdowns where AI-supported contributions did not result in a merged PR. By segmenting our dataset in this way, we surface distinct usage patterns, support comparative analysis across outcomes, and frame actionable insights for improving AI-assisted pull request workflows.

3.1.3 Rationale for Descriptive Analysis

Our quantitative analysis is primarily descriptive due to the heterogeneous and context-dependent nature of AI-assisted pull request interactions. Prior pull request studies have applied inferential models using stable outcome variables such as merge status, review latency, or patch size (Gousios et al. 2014; Tsay et al. 2014; Gousios et al. 2016). In contrast, categories such as PA, PN, and NE reflect qualitatively different forms of engagement, including partial reuse, conceptual guidance, and rejection after deliberation, which are not directly comparable across pull requests. These categories represent qualitatively distinct engagement outcomes that cannot be meaningfully modeled using uniform inferential assumptions. We therefore use descriptive statistics to characterize integration patterns and complement them with qualitative analysis to interpret developer decision-making. Accordingly, our study is exploratory in nature, aiming to characterize integration patterns and developer decision-making rather than to test causal hypotheses.

3.2 Dataset Construction and Processing

Figure 2 provides a visual representation of the dataset extraction and processing workflow. The following steps outline the methodology used to identify, extract, and classify ChatGPT interactions within pull requests, ensuring a systematic and reproducible approach to data collection.

3.2.1 Step 1: Identification of merged pull requests containing ChatGPT conversation shared links:

We build on the original DevGPT dataset (Xiao et al. 2023). This dataset contains nine snapshots collected from GitHub and Hacker News between July 27, 2023, and October 12,

2023, featuring commits, issues, discussions, pull requests, and threads where developers shared links to ChatGPT conversations. Our study focuses only on the merged and closed (unmerged) pull requests included in these snapshots. The initial DevGPT dataset comprises 98 merged pull requests and 39 unmerged (closed) pull requests.

Expanding the DevGPT Dataset To ensure a comprehensive and up-to-date analysis, we aimed to enlarge the dataset by including more recent instances of merged pull requests and those that are closed without merging but containing ChatGPT conversation links. Using GitHub API v3 (GitHub: GitHub REST API Documentation 2025), we replicated the original data collection methodology with modifications to accommodate data from October 13, 2023, to August 24, 2024. Our search parameters targeted pull requests that specifically included the keyword “<https://chat.openai.com/share>”. We used a regular expression (“[https://chat.openai.com/share/\[a-zA-Z0-9-\]{36}](https://chat.openai.com/share/[a-zA-Z0-9-]{36})”) to precisely extract ChatGPT links from pull request descriptions, commits, or review discussions.

This process identified 187 new merged PRs and 14 closed/unmerged PRs. We excluded only PRs whose ChatGPT share links were inactive (e.g., deleted or inaccessible). We did not filter repositories by popularity or overall activity, but required that each repository contain at least one PR with at least one review comment to ensure meaningful discussion context. The final dataset comprises 338 PRs from 255 unique repositories. Summary repository metrics (median stars = 58, median forks = 31, median watchers = 946) indicate that most projects are active and nontrivial; full distributions are presented in Section 4.1 (Fig. 3).

3.2.2 Step 2: Extracting Pull Requests and ChatGPT Patches

The pull request and ChatGPT patches are central for analysis in *RQ1*. Our dataset consists of links to pull requests, ChatGPT links, and patches (if any) extracted from the conversation. The process of extracting patches from ChatGPT conversations was not completely automated, especially in the extended dataset. We noted that OpenAI’s updated terms of service (as of November 14, 2023) prohibit users from automatically extracting data directly from their services (OpenAI: Terms of Use 2025). For this reason, we did not employ any automation or scraping tools on ChatGPT’s website or API. Instead, we manually converted each shared conversation link obtained in *Step 1* into a local HTML file and performed any necessary parsing only on these static offline copies local without making HTTP requests or automated interaction with OpenAI’s systems. Pull request patches were extracted using GitHub’s `/pulls/` endpoint and saved as “diff” files. At the end of this step, we had two sets of patches for each pull request: ChatGPT and pull request patches. This data served as input for the patch classification phase.

3.2.3 Step 3: Patch Classification

To answer *RQ1*, we needed a reliable way to determine whether and how ChatGPT-suggested patches were integrated into pull requests. The goal of this step is to classify each pull request into one of three integration outcomes based on how the AI-generated patch relates to the developer’s final code: Patch Applied (PA), Patch Not Applied (PN), or No Patch

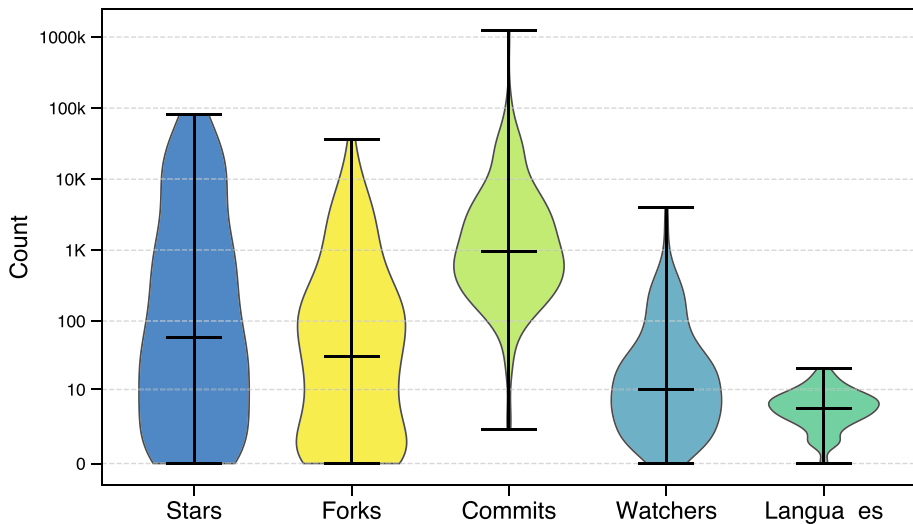


Fig. 3 Characteristics of the repositories curated in the dataset

Exists (NE). A pull request is labeled PA if ChatGPT-suggested code appears in the merged changes, PN if code was suggested but not incorporated, and NE if no code patch was suggested in the conversation. For brevity, we present only the core classification logic here and defer implementation details, validation procedures, and robustness analysis to Appendix A.

To enable this analysis at scale, we developed *PatchTrack*, an automated tool that detects and compares ChatGPT-generated code snippets with pull request diffs. The tool outputs a patch-level label indicating whether the ChatGPT code was directly applied, discarded, or absent, supporting large-scale empirical analysis of developer behavior under self-admitted ChatGPT usage. Patch classification is performed at the hunk level by comparing tokenized representations of ChatGPT snippets with added lines in pull request diffs.

Based on a manual validation study of 230 pull requests, we selected an n -gram size of $n = 1$, which achieved the best balance of accuracy, precision, and recall while capturing short and partial code fragments common in ChatGPT responses. Inter-rater agreement between the two authors yielded a Cohen's Kappa of 0.85, indicating strong reliability. Additional labels used to capture unsupported file types and processing failures are handled internally by the tool and described in Appendix A, along with detailed descriptions of *PatchTrack*'s design, normalization procedures, n -gram sensitivity analysis, performance metrics, error sources, and robustness checks.

3.2.4 Step 4: Pull Request Classification

The classification of pull requests in *Step 3* follows a straightforward approach. For each class, we aggregate its occurrences within a pull request. If the count of PA is greater than zero, the entire pull request is classified as PA. If there are no instances of PA but there are instances of PN, the pull request is classified as PN. If no ChatGPT code snippets were identified in *Step 3*, both PA and PN counts for this pull request will be zero, leading to an overall classification of NE. Pull requests that were closed without merging are classified

as CL, regardless of whether they contain ChatGPT-suggested patches, allowing us to analyze closure-specific factors in RQ4. The analysis provided by PatchTrack shows how the 645 ChatGPT-generated code snippets were compared with 3,486 pull request patches, resulting in precise classifications and valuable insights into the interaction between developers and AI-generated patches. While PatchTrack classifies each individual hunk, our analysis aggregates the results at the pull request level. This is because patch usage decisions (application, revision, rejection, or closure) are typically discussed and resolved within the PR context rather than at the level of individual snippets.

3.2.5 Step 5: Qualitative Analysis

To address RQ1–RQ4, we conducted a qualitative analysis of pull requests pre-processed and classified by PatchTrack. For RQs, we analyzed cases labeled PA, PN, NE, and CL, which represent scenarios where ChatGPT suggestions were adopted, modified, conceptually reused, or ultimately rejected. For each case, we examined the linked ChatGPT conversation, the surrounding GitHub discussion (up to three developer messages before and after the shared link), and relevant code diffs. This interaction window was selected based on pilot testing, which showed it typically captured sufficient context without including unrelated discussion. Code changes were also reviewed to assess alignment with AI suggestions.

We applied the Framework Method (Gale et al. 2013), proceeding through structured summarization, codebook development, and theme refinement. One author and a research assistant independently reviewed overlapping subsets of PR cases and generated structured summaries using a consistent template. Each summary captured the developer's problem, the ChatGPT recommendation, the developer's response, the resulting code outcome, and any reviewer feedback. To improve consistency and reduce fatigue for longer threads, we tested GPT-4-generated summary drafts. These were always reviewed and revised by a human analyst. In cases where the model introduced hallucinations or omitted relevant context, the human-authored version was prioritized. All summaries were then reviewed and finalized in discussion with a senior researcher to ensure agreement and interpretive alignment across cases.

All summaries were collaboratively coded and grouped into higher-level themes through card sorting, continuing the three-person team from the summarization phase (one author, a research assistant, and a senior researcher). Initial coding and grouping were performed by the first author and assistant, with discrepancies discussed and resolved in consensus meetings involving the second author and senior researcher. These discussions helped clarify the relationships between surface behaviors and underlying motivations, ensuring interpretive consistency. While we did not compute Cohen's Kappa due to the qualitative nature of the task, agreement was verified through iterative discussion and convergence on shared meanings. The final thematic structure was applied across the full dataset.¹

While our coding was primarily inductive, we consulted prior studies on pull request rejection and review dynamics (Soares et al. 2015; Gousios et al. 2014, 2016) to inform our general understanding. These provided background perspective but did not directly guide

¹The research assistant contributed only to manual analysis and theme sorting. They were not involved in study design, interpretation, or writing and therefore do not meet authorship criteria. This follows ACM's authorship policy: <https://www.acm.org/publications/policies/new-acm-policy-on-authorship>

our codes, as they do not account for the unique characteristics of AI-assisted contributions. Because AI-generated patches introduce distinct risks and social dynamics, such as hallucinated output, prompt ambiguity, developer over-reliance on AI, and uncertainty in trust attribution, we extended our codebook by open coding 20 representative cases to surface patterns grounded in the data. Examples of structured summaries and the code-to-theme mapping are included in our replication package (Ogenrwot and Businge 2025) to support transparency and reproducibility.

4 Results and Discussion

This section presents the key findings of our study. We begin by characterizing the repositories and pull requests that form the basis of our dataset. This provides essential context for interpreting how ChatGPT-generated patches are used in real-world development workflows. We then answer each of our research questions in turn, starting with RQ1, which quantifies the integration of ChatGPT-generated code across pull requests.

4.1 Repository Characteristics

The 338 pull requests in our dataset come from 255 unique GitHub repositories that vary widely in maturity, popularity, and technical focus. Figure 3 summarizes core repository attributes. Popularity indicators are long-tailed: stars range from 0 to ~81,000 (median: 58) and forks from 0 to 36,655 (median: 31). Commit histories are likewise highly skewed, with counts peaking above 1.2 million (median: 946). The language panel further indicates coverage across diverse implementation ecosystems. These attributes indicate that our dataset includes both niche and high-profile projects. Prior studies have used similar repository-level metrics to capture ecosystem diversity and developer participation (Weeraddana et al. 2023; Wang et al. 2023; Businge et al. 2022).

In terms of language diversity, the dataset covers 123 distinct programming languages. The most frequently observed languages include Shell (85.5%), JavaScript (82.0%), HTML (62.0%), TypeScript (50.0%), Python (54.1%), and CSS (54.0%), reflecting a strong representation of web development and scripting tasks. Infrastructure and configuration files are also well represented, such as Dockerfile (49.0%) and Makefile (33.0%). Compiled and system-level languages such as Java (19.0%), C (18.4%), and C++ (14.5%) are present but less dominant.

While this linguistic variety highlights the breadth of our dataset, its distribution diverges from GitHub's overall language trends. According to GitHub's 2024 Octoverse report (GitHub: The State of Open Source 2024), Python has surpassed JavaScript as the most used language, with Java, C, and C++ also ranking among the top. In contrast, our dataset overrepresents Shell, JavaScript, and other front-end and scripting technologies. This suggests that developers are more likely to use ChatGPT in tasks that involve scripting, web development, and configuration management, domains that favor automation and well-defined coding patterns. By contrast, areas that demand deeper domain knowledge or complex architectural decisions (e.g., backend systems, data science) may be less conducive to AI-assisted contributions.

These findings suggest that current ChatGPT usage in pull request workflows is concentrated in specific development contexts. Broader conclusions about generative AI adoption will require datasets with greater alignment to GitHub's full language ecosystem. Future work could address this by stratifying repositories according to GitHub language distributions and exploring AI-supported development practices across a wider range of software domains.

4.2 RQ1: Classification of Patch Use in Pull Requests

RQ1: How are ChatGPT-generated patches used and integrated in pull requests under self-admitted ChatGPT usage, as observed through patch-level classification?

This research question investigates the distribution of ChatGPT-assisted pull requests across three integration categories: Patch Applied (PA), Patch Not Applied (PN), and No Patch Exists (NE). Using the final configuration of PatchTrack ($n = 1$), we analyzed 285 pull requests, resulting in the following classification:

- **Patch Applied (PA)**: 116 cases (40.7%)
- **Patch Not Applied (PN)**: 63 cases (22.1%)
- **No Patch Exists (NE)**: 106 cases (37.2%)

Figure 4 visualizes this distribution. While many developers adopted ChatGPT-generated patches, others rejected or heavily modified them, and in some cases, ChatGPT did not generate a patch at all. These outcomes reflect varying levels of trust, relevance, and integration effort, which we explore further in RQ2–RQ4.

To assess the sensitivity of these results to potential PatchTrack misclassification, we recomputed the main RQ1 statistics using only the manually validated subset of 230 pull requests. The relative distribution and ordering of the PA, PN, and NE classifications remained consistent with those observed in the full dataset, and no classification shifted from minority to majority. The median integration rate for PA cases also remained unchanged.

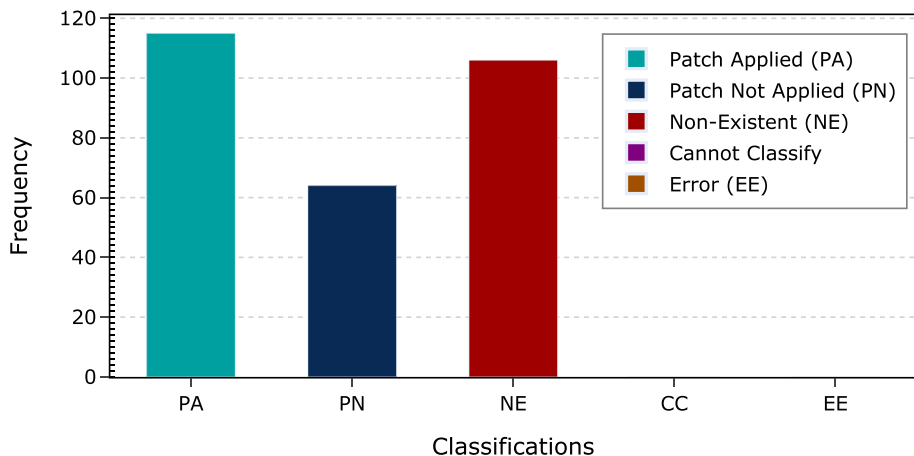


Fig. 4 Distribution of patch classification results

This indicates that the aggregate patterns reported for RQ1 are robust to possible classification noise.

4.2.1 Patch Classification

We analyzed 645 ChatGPT-generated code snippets and 3,486 patches across 285 pull requests. Each snippet was compared against all patches in its corresponding PR using *PatchTrack*, which employs token-level matching to determine integration. This process enabled classification into *PA*, *PN*, or *NE*, as described in Section 3.2.3.

As validated in our methodology (Section 3.2.3), using an *n*-gram size of $n = 1$ yielded the highest accuracy, precision, recall, and F1 score, due to its sensitivity to short code fragments. This configuration was used for all reported results. Figure 4 summarizes the resulting distribution of PRs across the three integration categories.

Percentage of ChatGPT Patches Applied To assess how much of ChatGPT’s suggested code is integrated into pull requests, we compute the percentage of ChatGPT patches applied. This metric helps evaluate the degree to which developers adopt AI-generated suggestions when resolving issues or adding features. By linking overlap percentage to classification labels, we can systematically evaluate developer adoption behavior and determine how self-admitted ChatGPT usage translates into actual code integration. For example, a high ratio indicates direct reuse of ChatGPT’s output, while a low ratio may suggest heavy modification or rejection. This directly supports our goal of classifying integration outcomes. We use Jaccard’s containment ratio 1 to quantify the overlap between tokens in the ChatGPT-suggested patch (t_x) and the final integrated code (t_y):

$$Jacc(t_x, t_y) = \frac{|t_x \cap t_y|}{|t_x|} \times 100 \quad (1)$$

Here, t_x and t_y are token streams from ChatGPT and GitHub patches, respectively. This formulation, adopted from the *Redebug* study (Jang et al. 2012), measures the proportion of ChatGPT’s code that appears in the final merged pull request. Figure 5 illustrates the distribution of integration percentages for the classification *PA*. The median integration rate is 25%, indicating that, on average, a quarter of the ChatGPT-suggested code is adopted in merged pull requests.

Figure 5 reports the proportion of ChatGPT-suggested code that overlaps with the final pull request. This metric captures the degree of reuse between AI-generated and integrated code, but does not distinguish whether overlapping lines correspond to newly added code or modifications of existing code. As such, the figure should be interpreted as indicating reuse intensity rather than the semantic nature of the changes.

Developers integrate ChatGPT patches with varying degrees of modification, assessing their relevance based on project needs. While some adopt AI-generated patches wholesale, others incorporate only selective portions. This variation reflects how developers balance AI-assisted suggestions with project-specific constraints such as maintainability, readability, and adherence to coding standards. The distribution of integration percentages, as shown in Fig. 5, highlights this diversity in adoption, with some pull requests fully incorporating ChatGPT-generated code, while others retain only fragments. The presence of outliers in the

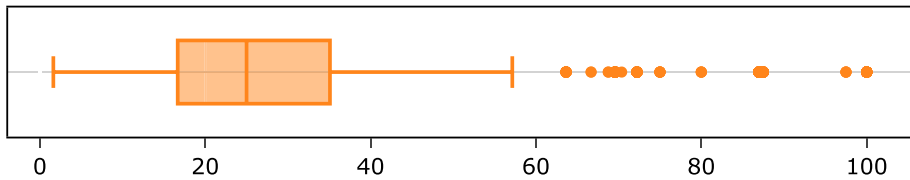


Fig. 5 Boxplot showing the percentage of lines of code from the ChatGPT patches that are integrated into pull requests

distribution underscores the exception rather than the norm: while a few developers fully adopt ChatGPT-generated patches, the majority incorporate only parts of the suggestions, often modifying them extensively to fit project-specific requirements.

The median integration rate of 25% (Q2) suggests that full adoption of ChatGPT-generated patches is uncommon, with most pull requests retaining only a fraction of the AI-suggested code. The third quartile (Q3) at 38% further reinforces that even in cases with higher integration, significant modifications are often required. These findings indicate that developers frequently treat the ChatGPT output as a **starting point rather than a final implementation**, selectively incorporating code that aligns with project needs.

To better understand this variation, we selected four pull requests representing different levels of integration, with one pull request chosen from each quartile: (i) *low* (0–25%], where only minor elements of the suggested patch were retained, often requiring substantial modifications or alternative implementations; (ii) *lower-median* (25–50%], where developers incorporated portions of the patch but refined it significantly; (iii) *upper-median* (50–75%], where a larger portion of the patch was adopted with some modifications; and (iv) *high* (75–100%], where the AI-generated patch was largely accepted with minimal changes. Each quartile follows the standard convention where the lower bound is excluded and the upper bound is included. These cases were selected through stratified random sampling to ensure balanced representation across integration levels. By analyzing these strata, we illustrate real-world scenarios of ChatGPT-assisted patch integration, shedding light on why some patches are fully applied while others are selectively merged or heavily revised. These examples are illustrative and not intended for theory-building, so theoretical saturation was not a goal.

4.2.2 Qualitative Analysis – RQ1

To complement the quantitative analysis of patch adoption rates, we conducted a thematic analysis of the validated PA cases. Although 90 cases were initially sampled, four were later reclassified as PN and three from PN to PA, resulting in a final set of 89 PA cases (see Section 3.2.3 for details on misclassifications). Coding followed the structured framework described in our methodology. Agreement was reached through coder discussion and convergence on six themes.

RQ1 Qualitative Themes To support interpretation of the theme distributions reported below, we briefly summarize the six qualitative themes used to code ChatGPT’s contribution in pull requests. These themes capture distinct modes of developer engagement with AI output, ranging from direct code reuse (*Direct Implementation*), partial or selective reuse

(*Selective Extraction*, *Structural Integration*), and iterative modification (*Iterative Refinement*), to constraint-driven adaptation (*Constraint Handling*) and non-code influence such as conceptual guidance or documentation support (*Knowledge Support*). Full operational definitions and coding criteria are provided in Appendix B.

Table 1 summarizes how themes distribute across integration ranges. For brevity, we present one representative example per quartile below. Each instance is labeled PA-X (where X is the instance number). Full case summaries, coding assignments, pull request links, and ChatGPT-developer conversation links are available in the replication package (Ogenrwot and Businge 2025).

✓Low Integration (0–25%) This quartile (34 cases) is characterized by *Structural Integration* (8 cases) and *Selective Extraction* (8 cases) as the most distinctive behaviors for minimal carryover of AI code, even though *Iterative Refinement* appears slightly more often numerically (10 cases). We emphasize *Structural Integration* and *Selective Extraction* here because they most clearly explain *why* integration remained low: developers either reshaped ChatGPT’s output to fit project frameworks or extracted only fragments of functionality, discarding the rest. These patterns reflect how AI output typically required substantial reshaping before it could be meaningfully integrated in this range (counts from Table 1). Example 1 (PA-27)–Case ID: PA-27; Theme: *Structural Integration* illustrates this pattern: In pull request #185 (GitHub pull request 2024a) of the pokt-network/poktrol1 repository, the developer asked ChatGPT² if Go had a built-in function to remove leading spaces from multiline strings while maintaining code readability. ChatGPT suggested an approach using `strings.Split`, `strings.TrimLeft`, and `strings.Join` to process each line and remove leading spaces. The developer then requested a refined solution that encapsulated this logic into a reusable helper function, `trimString`, to improve readability. ChatGPT provided a full Go program, but only its helper function was integrated. Of the suggested 31 LOC, just 7 LOC (22.6%) were adopted. The minimal adoption reflects the developer’s selective integration of only the core functionality needed, as the existing project structure did not require a complete rewrite.

✓Lower-median Integration (25–50%) This quartile (25 cases) is dominated by *Selective Extraction* (9 cases) and *Structural Integration* (9 cases), with *Constraint Handling* and *Iterative Refinement* also present but less frequent (counts from Table 1). Developers in this range often incorporated substantial portions of ChatGPT’s suggestions, but not wholesale:

Table 1 Summary of themes across integration ranges

Theme	(0–25%]	(25–50%]	(50–75%]	(75–100%]	Total
Constraint Handling	4	3	3	0	10
Direct Implementation	0	0	0	3	3
Iterative Refinement	10	3	7	6	26
Knowledge Support	4	1	5	3	12
Selective Extraction	8	9	0	1	18
Structural Integration	8	9	1	1	19
Total	34	25	16	14	89

²Example 1 link to ChatGPT conversation – Accessed: 2025-02-17

they selectively retained useful fragments or adapted the AI output to align with coding conventions. Integration percentages therefore remained below 50%, reflecting the tendency to treat ChatGPT's patch as scaffolding rather than final code. Example PA-36; Theme: *Selective Extraction* illustrates this pattern: In pull request #7123 (GitHub pull request 2024b) of the Mudlet/Mudlet repository, a crash occurred when double-clicking a word in TTextEdit, traced to an *out-of-bounds access* in text-selection logic. After reviewing the *stack trace*, the developer consulted ChatGPT³, which proposed a structured patch: (i) add boundary checks for `yind`, (ii) break loops when indices become invalid, and (iii) refactor selection logic for robustness. The 39-LOC patch addressed the crash, but only 18 LOC (46.2%) were integrated. A reviewer suggested simplifying the control flow by *negating conditions to reduce indentation*, prompting further edits. The adopted code retained the essential boundary checks while discarding or refactoring the rest, showing how developers extracted only the useful fragments of ChatGPT's logic while reshaping them to improve readability and project alignment.

✓Upper-median Integration (50–75%) This quartile (16 cases) is characterized by *Iterative Refinement* (7 cases) as the most common theme, followed by *Constraint Handling* (3 cases) and *Knowledge Support* (5 cases) (counts from Table 1). Developers in this range integrated a majority of ChatGPT's suggestions, but usually with targeted refinements rather than direct adoption. Reviewer input often drove these refinements, such as clarifying class names, reorganizing logic, or improving readability. Integration percentages in this quartile therefore reflect substantial reuse of ChatGPT code, tempered by human oversight to align with project conventions. Example PA-73; Theme: *Iterative Refinement* illustrates this pattern: Pull request #279 (GitHub pull request 2024c) in the nylas/nylas-python repository introduced support for a new “free-busy” API endpoint in the Nylas SDK, enabling users to check calendar availability for specific email addresses. A key challenge was designing a suitable data model to represent the API response, which could include both availability data and error messages. To address this, the developer consulted ChatGPT⁴ for guidance on structuring Python dataclasses. ChatGPT suggested: (i) defining separate data models for distinct response components, (ii) using Python's `dataclass` module for maintainability, and (iii) employing `Union` types to handle multiple object types within a list. The proposed patch contained 25 LOC, of which 15 LOC (60%) were integrated. The main adjustment involved renaming the `error` class to `freeBusyError` for clarity, following reviewer feedback. This case shows how AI-generated code can align closely with project needs while still requiring human-led refinements to enhance readability and maintainability.

✓High Integration (75–100%) This quartile (14 cases) is dominated by *Direct Implementation* (3 cases) alongside instances of *Iterative Refinement* (6 cases) and *Knowledge Support* (3 cases) (counts from Table 1). Developers in this range adopted the ChatGPT output with little modification, often because AI-generated code already aligned with project requirements. When changes occurred, they were minor and stylistic rather than structural. These cases highlight scenarios in which ChatGPT produced production-ready code that required minimal oversight before integration. Example PA-82; Theme: *Direct Implementation* illus-

³Example 2 link to ChatGPT conversation – Accessed: 2025-02-17

⁴Example 3 link to ChatGPT conversation – Accessed: 2025-02-17

trates this pattern: Pull request #2059 (GitHub pull request [2024d](#)) of the `alshedivat/al-folio` repository extended a Jekyll plugin to support fetching posts from a manually specified list of URLs in addition to an RSS feed. Initially, the plugin retrieved posts only via RSS, but the developer aimed to enhance functionality by incorporating direct URL input. To achieve this, they consulted ChatGPT⁵, focusing on three key aspects: (i) metadata extraction, (ii) correct handling of published dates, and (iii) improved logging readability. ChatGPT refactored the Ruby plugin to introduce modular functions, ensured robust date handling with `Time` objects, improved log clarity through `puts`, and extracted date parsing into a dedicated `parse_published_date` function. The patch consisted of 19 LOC, all of which were integrated as-is (100%), since the output aligned perfectly with project needs. The developer's enthusiasm, expressed at the end of the interaction ("*Awesome! I have a feeling we'll write a lot of code together from now on :)*"), underscores how ChatGPT's response was fully applicable without requiring further adjustments.

4.2.3 Discussion and Implications – RQ1

Our findings show that while ChatGPT-generated patches provide value, full adoption remains uncommon, with a median integration rate of 25%. Developers most often treated ChatGPT's output as a *starting point* rather than a final implementation, selectively integrating elements that aligned with project-specific needs. Thematic analysis of 89 validated PA cases revealed four dominant integration patterns across quartiles: *Selective Extraction*, *Structural Integration*, *Iterative Refinement*, and *Direct Implementation*. These patterns explain not only how much of ChatGPT's code was reused but also why integration remained partial or complete.

In low-integration cases (0–25%), developers retained only minor portions of ChatGPT's suggestions, typically extracting fragments or reshaping code to fit project frameworks. Moderate integration cases (25–75%) revealed substantial reuse, but usually with targeted refinements, such as simplifying control flow, renaming classes, or aligning with architectural constraints, often informed by reviewer feedback. High-integration cases (75–100%) occurred when ChatGPT's output already conformed to project conventions, enabling near-direct adoption with minimal oversight. These variations highlight that ChatGPT accelerates exploration but rarely produces production-ready patches without human adaptation.

The predominance of low- and moderate-integration cases underscores ongoing challenges in AI-assisted development, including misalignment with project coding styles, maintainability concerns, and trust calibration. This suggests a need for AI tools that incorporate greater contextual awareness, generating code that requires less reshaping. At the same time, high-integration cases demonstrate AI's potential to streamline the pull request process when outputs align with project expectations, reducing the effort of implementation.

Importantly, our study focuses on self-admitted ChatGPT usage (SACU), which means that all the cases analyzed involve developers explicitly stating their use of ChatGPT within pull request discussions. This ensures transparency in identifying AI-assisted contributions, but also introduces a limitation: *we do not capture instances where ChatGPT was used privately or without disclosure*. The question of whether undisclosed ChatGPT usage follows a different pattern of patch integration, which could influence the observed distribution of PA, PN, and NE cases remains open.

⁵ [Example 4 link to ChatGPT conversation](#) – Accessed: 2025-02-17

These findings resonate with the longstanding vision of the “Programmer’s Apprentice” (Rich and Waters 1990), positioning ChatGPT not as an autonomous coder but as a junior collaborator that supports debugging, scaffolding, and refinement while leaving final decisions to developers. By classifying adoption behaviors across granular integration levels, we show that ChatGPT functions primarily as a decision-support tool, shaping but not replacing human judgment in pull request workflows.

Finally, this analysis establishes a foundation for the subsequent research questions. While RQ1 quantifies *how* ChatGPT-generated patches are integrated and characterizes adoption patterns, RQ2–RQ4 explore the complementary cases: why patches are not applied (PN), why no patches are suggested (NE), and why some pull requests close despite ChatGPT’s involvement (CL).

Connection to Prior Work Our findings complement and extend prior studies on AI-assisted code generation (Huang et al. 2024; Menzies and Pecheur 2020; Ozpolat et al. 2023), code review (Tufano et al. 2024b; Xiao et al. 2024), and pull request workflows (Gousios et al. 2014; Tsay et al. 2014; Gousios et al. 2016). While prior work has quantified productivity benefits or patch acceptance (Jiang et al. 2023; Guo et al. 2024; Deng et al. 2024), few studies have examined how AI-generated patches are actually evaluated, reshaped, or rejected within the collaborative dynamics of pull requests. Our contribution builds on recent efforts to capture patch-level decisions (Grewal et al. 2024; Siddiq et al. 2024), but goes further by stratifying cases across integration quartiles and classifying behavioral themes. In doing so, we show that generative AI functions less as a deterministic patch source and more as a catalyst for developer reasoning and selective adoption.

Summary of RQ1 Results: *ChatGPT-generated patches are seldom adopted wholesale. The median integration rate is 25%, with adoption patterns varying across quartiles. Our qualitative analysis of 89 cases revealed six recurring themes, with most developers engaging in **iterative refinement**, **structural integration**, or **selective extraction** rather than direct implementation. These findings show that developers consistently treat AI output as a starting point, adapting it to project-specific constraints and reviewer feedback. PatchTrack enabled this analysis by classifying patch use with 97.5% accuracy and surfacing distinct adoption patterns. Together, these insights motivate RQ2–RQ4, which explore why patches are rejected, why some PRs lack AI-suggested patches, and why others close without adoption despite ChatGPT involvement.*

4.3 Results and Discussion – RQ2, RQ3 & RQ4

This section builds on the validated dataset from RQ1, where 230 pull requests were manually reviewed to assess patch integration outcomes: 90 PA, 56 PN, and 84 NE. We now examine the PN and NE subsets to answer RQ2 and RQ3. For RQ4, we independently sampled 47 closed pull requests from a total of 53 using a 95% confidence level and a 5% margin of error.

Throughout this analysis, we do not interpret patch non-adoption (PN) or pull request closure (CL) as evidence of low code quality; rather, we treat these outcomes as reflecting contextual constraints, integration effort, and maintainer decision-making during review.

Thematic coding was conducted using a structured framework based on our research questions. While we did not compute Cohen's Kappa due to the interpretive nature of the task, agreement was reached through coder discussion and convergence on final themes. Details of the coding procedure and reviewer roles are described in Section 3.2, Step 5.

Qualitative Themes for RQ2–RQ4 Across RQ2, RQ3, and RQ4, we identify recurring themes that characterize how ChatGPT influenced developer decision-making when patches were not applied (PN), no patches were generated (NE), or pull requests were closed (CL). For each research question, we introduce a brief inline characterization of the relevant themes before presenting illustrative cases, focusing on how ChatGPT shaped methodology, design choices, documentation and communication, debugging and optimization, or was constrained by technical, procedural, or scope-related factors. Full operational definitions for all themes are provided in Appendix C, with detailed coding criteria and additional examples available in the replication package (Ogenrwot and Businge 2025).

For brevity, we present one representative instance per theme in the results sections. Additional examples and detailed summaries are available in the replication package (Ogenrwot and Businge 2025), which includes all summaries, identified themes, pull request links, and corresponding ChatGPT-developer conversation links. Each instance is labeled with a unique identifier: PN-X (X being the instance number) for patches not applied, NE-X for cases where no patches were suggested, and CL-X for closed pull requests. These identifiers support cross-referencing with the replication package.

We first present the results, followed by the implications at the end of each research question. In the results, we provide concrete examples and notable excerpts from pull request discussions. Themes are highlighted in **bold fonts**, conversations are presented in *italics*, keywords related to a particular theme are underlined, and longer messages are abbreviated with ellipses [...]. For each theme, we discuss the implications for practitioners and/or researchers using the icons  and , respectively.

4.4 Results and Discussion – RQ2

Why are ChatGPT-suggested patches in conversations not directly integrated (PN) into the pull request, and how do developers use ChatGPT's suggestions in their workflow under the self-admitted ChatGPT usage? In our analysis, PatchTrack initially classified 63 instances as patch not applied (PN). To validate the accuracy of this classification, we manually reviewed a sample of 55 PN instances, selected using a 95% confidence level with a 5% margin of error, as detailed in Section 3.2.3. During this review, we identified 3 false positives that were incorrectly labeled as PN and 4 false negatives from the patch applied (PA) set that should have been categorized as PN. After applying these corrections, we obtained a final set of 56 validated PN instances for thematic analysis. The analysis of these 56 instances revealed six distinct themes, as illustrated in Fig. 6.

 **Adaptation and tailored solutions:** This emerged as the most prevalent theme, with 17 of 56 cases showing that developers adapted the conceptual insights of ChatGPT to fit specific project requirements. For example, in pull request #12 (GitHub pull request 2023c)

– PN-19, a developer sought a regular expression from ChatGPT to match the ULID format in the `laravel-json-api/core` project. The developer initially considered ChatGPT’s regex: *“Like I said, I didn’t find an ‘official’ regex for the ULID. ChatGPT gave me this: [...]”*. However, rather than directly applying the AI-generated solution, the developer aligned the implementation with Laravel’s native method for handling ULIDs. This decision was influenced by the comment of a reviewer: *“I think we should match what Laravel does” for route regexes for ULIDs. Looking at this docs: [...]*. The author acknowledged the feedback: *“[...] good idea [...]. Change made!”* and updated the code to ensure consistency within the Laravel ecosystem. This example highlights how developers use ChatGPT as a starting point, but ultimately refine AI-generated suggestions to conform to established coding conventions and project-specific constraints.

✓Technical limitations and preferences This was the second most prominent theme, with 9 of 56 instances demonstrating how ChatGPT’s patches were not applied due to technical constraints or developer preferences for alternative solutions. In these cases, developers faced trade-offs in implementation strategies, either due to limitations in the existing system or subjective choices regarding the best approach to a problem. A representative example is pull request #171 (GitHub pull request [2023d](#)) within the `ory/elements` project (PN-11), where developers attempted to disable a button when submitting the form. A reviewer sought ChatGPT’s input on a possible fix, asking: *“Wouldn’t the fix just be to change click to submit? [...]”* However, another reviewer pointed out a critical issue with this approach: *“This doesn’t actually work. The reason is that the click event fires before the submit event [...] but IMO we should just get this right [...]”* The pull request author ultimately acknowledged that resolving the issue properly would require additional time and effort, stating: *“[...] just don’t want to spend time on getting this right, right now [...]”* Upon further examination, we found that the proposed solution was reverted because disabling the button caused unintended side effects, such as stopping event propagation. Given the

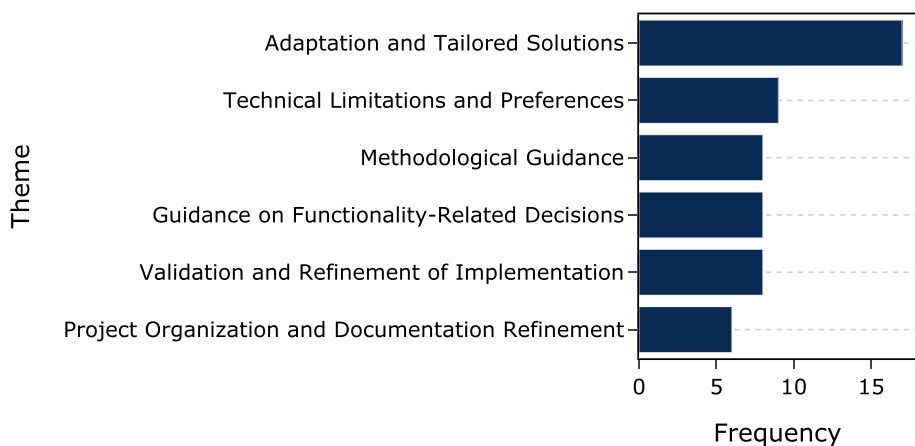


Fig. 6 Showing the distribution of themes in the patch not applied (PN) class

minor impact of the issue, the team decided to focus on more pressing updates rather than implementing a workaround.

✓Methodological Guidance We identified 8 of the 56 instances in which ChatGPT influenced developers' decision-making strategies by suggesting optimizations or best practices rather than providing direct fixes. For example, in pull request #5063 (GitHub pull request [2023e](#)) within the `darklang/dark` project, a developer attempted to resolve an issue related to function composition. A reviewer, finding the initial implementation overly complex, consulted ChatGPT, which suggested a more concise approach. The reviewer commented: *"While this works, it feels like overkill. I asked ChatGPT about how best to do this, and it suggested using (int)c. That seems to me like it would work. ChatGPT-link[...]"*. This led the pull request author to reassess the solution, ultimately adopting a simplified version influenced by ChatGPT's recommendation.

✓Guidance on Functionality-Related Decisions Developers often seek to modify or enhance specific functionality in software projects, often consulting ChatGPT for insights on best practices or alternative implementations. This theme captures instances where ChatGPT's recommendations influenced functionality-related decisions, even when no direct patches were applied. From our dataset, we identified 8 of the 56 instances in this category. A representative example is pull request #522 (GitHub pull request [2023f](#)) within the `sveltejs/learn.svelte.dev` repository (PN-13). In this case, the pull request author sought guidance on the correct ARIA role for an interactive SVG acting as a pointer in a Svelte application. The conversation focused on whether to use `"role=none"` or `"role=presentation."` ChatGPT provided contextual insights on ARIA accessibility, suggesting that `"role=presentation"` would be more appropriate based on the functionality of the SVGs. The pull request description explicitly references ChatGPT's recommendation: *"presentation" might be better suited here according to ChatGPT: [...]*. The integrator acknowledged the contribution with a brief confirmation: *"Thank you!"* and merged the pull request. This case illustrates how ChatGPT's suggestions can shape design decisions and accessibility improvements, with maintainers providing the final validation before integration.

✓Validation and Refinement of Implementation Developers often use ChatGPT to *validate their reasoning and refine their decision-making process* rather than applying AI-generated patches directly. In 8 of 56 instances, the interactions focused on seeking conceptual clarification or evaluating alternative implementations before finalizing a solution. This indicates that AI tools play a role not only in code generation, but also in *developer cognition and reasoning during pull request workflows*. A representative example is pull request #5058 (GitHub pull request [2023g](#)) – PN-28, where a developer was considering implementing the `List.filter` function using recursion versus `List.fold` in a functional programming context. A reviewer commented: *"I was thinking about whether recursive implementations were more appropriate here, since this can also be implemented with List.fold. [...] You might enjoy what ChatGPT told me about it: [...]"* This prompted further discussion about theoretical best practices in functional programming, highlighting how ChatGPT is used as an advisory tool rather than a direct source of patches. Ultimately, rather

than replacing developer decision-making, ChatGPT functioned as a catalyst for discussion and refinement within the pull request workflow.

✓Project Organization and Documentation Refinement Beyond code modifications, developers frequently use ChatGPT to help improve project organization, improve documentation, and optimize workflow structures. This theme captures instances where ChatGPT's suggestions influenced non-code contributions, such as file management and documentation strategies, rather than direct patch implementations. From our dataset, we identified 6 of the 56 instances in which ChatGPT was consulted to streamline internal project structures. A representative case is pull request #37 (GitHub pull request [2023h](#)) within the `Banana-pus/nana-core` project (PN-2), where the developer sought an efficient way to manage file renaming within a pull request. The ChatGPT prompt specified the issue: *"in a GitHub pull request, I have renamed a file. In the diff viewer, it shows one file deleted and another added, but I instead want to show the file as renamed."* ChatGPT suggested a structured file management approach to ensure that Git detected the renaming action instead of treating it as deletion and addition. The developer applied this suggestion, improving repository maintainability and commit traceability. No code patch was applied since the pull request focused on internal organization rather than functional modifications.

💡 Summary of RQ2 Results: *The analysis of 56 PN instances identified six main reasons why ChatGPT-suggested patches were not directly integrated: adaptation to project needs, technical limitations, methodological guidance, functionality-related decisions, validation and refinement of implementation, and documentation improvements. Developers frequently adapted AI-generated suggestions rather than applying them verbatim, often modifying them to fit project-specific constraints or replacing them with more suitable alternatives. In some cases, patches were rejected due to system limitations or developer preferences. In addition, developers use ChatGPT to validate ideas, explore alternative approaches, or refine documentation and project structure rather than directly integrating its generated code. These findings indicate that while ChatGPT plays a role in decision-making within pull request workflows, its suggestions are typically subject to critical evaluation and transformation before adoption.*

4.4.1 Implications for RQ2

While RQ1 revealed that developers rarely integrate ChatGPT patches verbatim, our RQ2 analysis adds a crucial nuance: even when patches are not applied (PN), ChatGPT influences developer decisions in meaningful ways. In these cases, developers assess suggestions critically and often extract conceptual guidance, architectural direction, or documentation support. This extends prior work (Grewal et al. [2024](#); Siddiq et al. [2024](#)), which focused on patch acceptance rates, by showing that AI-generated code can shape developer thinking and workflow even when it is ultimately not merged.

Our contribution also complements recent studies examining the limitations of generative AI in software engineering. While prior work has investigated correctness risks in AI-generated code (Tanzil et al. [2024](#)), cross-tool variability in patch quality (Li et al. [2024](#)), and the use of AI-generated metadata in PRs (Xiao et al. [2024](#)), our findings offer a behav-

ioral lens into how developers evaluate and selectively reuse AI suggestions. Additionally, we build on research into multi-hunk patch challenges (Nashid et al. 2025) and automation limitations in code review (Tufano et al. 2024b) by illustrating the human-in-the-loop decision-making that occurs when AI-generated patches are assessed but not applied.

Recent evaluations have highlighted limitations in LLMs' ability to reason about and adapt to project-specific constraints (Moumoula et al. 2024; Ouédraogo et al. 2024; Chen et al. 2024). These limitations likely contribute to patch rejection or modification. Our findings emphasize the need for AI systems that are better aligned with project context, coding conventions, and architectural patterns, an area that remains underexplored in current tooling.

☐ For developers Developers use ChatGPT's suggestions as conceptual starting points rather than direct solutions, often modifying or rejecting them based on project-specific requirements. This underscores the need for structured validation when using AI-assisted patches. Since ChatGPT-generated code may not always be directly compatible with existing architectures or coding styles, developers should critically assess AI suggestions within the context of system dependencies, maintainability goals, and pull request review feedback before applying them. This finding suggests that AI-assisted patch adoption can be improved by establishing clearer internal guidelines on when and how to incorporate ChatGPT's code into ongoing development workflows.

Beyond direct code generation, ChatGPT influences how developers refine and document their implementation strategies. Our results show that even when ChatGPT's patches are not applied, developers use AI-generated insights to improve documentation clarity, enforce project-specific conventions, and inform architectural decisions. This suggests that teams can benefit from integrating AI tools into pull request discussions, even when patches are not used directly, by treating them as a mechanism to refine implementation details and improve project documentation.

Q For Researchers These findings highlight a crucial research gap to understand how AI-generated suggestions are evaluated, modified, or dismissed prior to pull request integration. Future research should analyze how developers iteratively refine AI-assisted patches, distinguishing between modifications that preserve AI-generated logic versus those that entirely rework AI suggestions. Studying decision patterns, such as why some ChatGPT-suggested patches are partially modified while others are rejected, could provide insights into when AI-generated solutions are most effective.

In addition, improving AI-generated patches requires more context-aware recommendation models. Current AI systems lack fine-grained awareness of project-specific constraints, repository structures, and architectural dependencies, leading to suggestions that are syntactically valid but not semantically useful. Future research could explore adaptive AI models that learn from previous pull request discussions and historical development patterns, ensuring that AI-suggested patches align with specific project norms and are more seamlessly integrated into pull request workflows. This refinement would help bridge the gap between AI-assisted coding and real-world software engineering practices.

4.5 Results & Discussion – RQ3

Under what circumstances does ChatGPT not generate patches in developer interactions (no existing patch-NE), and how do developers use its responses under the self-admitted ChatGPT usage?

As shown in Table 3, PatchTrack classified 106 pull request instances as NE. For our analysis, we manually validated and examined a representative sample of 84 instances, as described in Section 3.2.3. From the generated summaries of the 84 NE instances, the employed card sorting analysis yielded four distinct themes, as illustrated in Fig. 7. The identified themes are: Conceptual Guidance & Theoretical Advice, Documentation, Communication & Translation, Education and Knowledge Sharing, and Debugging & Optimization Strategies.

✓Conceptual Guidance & Theoretical Advice This was the most prevalent theme, with 44 of 84 instances where ChatGPT did not suggest patches but instead guided pull request decision-making. Developers refined code based on programming concepts, design principles and best practices, improving readability, maintainability, and overall design. For example, in pull request #1061 (GitHub pull request 2023i) within the `codecrafters-io/frontend` project (NE-36), a reviewer used ChatGPT to critique a variable’s naming convention, stating: *“Bad naming, this should be something like selected-PricingFrequency. Reasoning: [ChatGPT link]”* The pull request author acknowledged the issue and noted: *“[...] we don’t seem to be getting better with naming [...] talk to ChatGPT about it [...] we need to pay extra attention to it.”* Originally, the variable name `pricing-FrequencyClicked` implied an action (clicking) rather than a state (selected frequency). After reviewing ChatGPT’s feedback, the developer renamed it to `selectedPricingFrequency`, improving clarity. While no code snippet was suggested, ChatGPT’s advice directly influenced the pull request decision, refining naming conventions. The absence of a patch did not hinder decision-making but encouraged refinement. Reviewers used ChatGPT’s insights to justify changes, while maintainers ensured clarity before merging. These interactions shifted pull request discussions from issue resolution to proactive quality improvements, reinforcing AI’s role in better development practices.

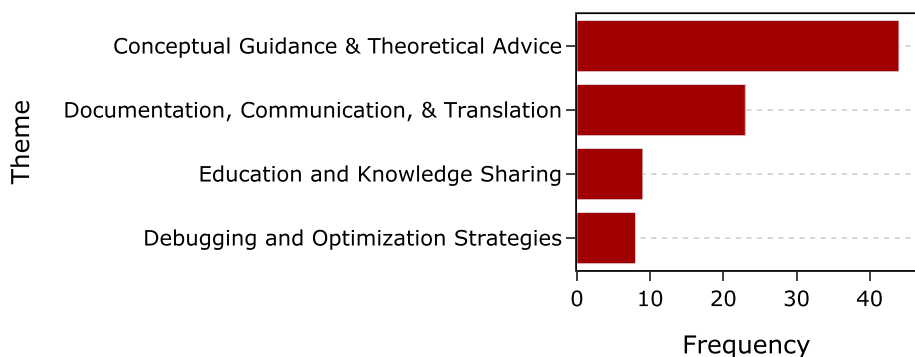


Fig. 7 Showing the distribution of themes in the No patch generated class

✓Documentation, Communication & Translation In 23 of the 84 instances, the discussions centered on refining the documentation, improving communication and ensuring accurate localization. Although no code snippets were generated, ChatGPT's responses helped developers refine explanations, adjust terminology, and align documentation with project standards before merging changes. A key example is pull request #2230 (GitHub pull request [2023j](#)) from the `faker-js/faker` project (NE-8), where a reviewer suggested re-writing a comment to be more positive and inclusive. The original comment stated: *"It would not be a good idea to have an English-only fallback list, because the Esper-anto community is very diverse and international."* To improve the phrasing, the reviewer consulted ChatGPT, who suggested multiple re-wordings. The final version was collaboratively adjusted and implemented, with the pull request author noting: *"Thank you for your co-mment, I used this suggestion!"* Although ChatGPT did not provide a code patch, its response influenced pull request decision-making by shaping documentation revisions before merging. AI-assisted refinements helped drive consensus in pull request reviews, leading to clearer and more inclusive communication. Although ChatGPT did not generate a code patch, its response influenced pull request decision-making by reinspecting documentation before merging. AI-assisted suggestions helped reviewers justify edits and maintainers ensure clarity, shifting pull request discussions from minor wording adjustments to broader quality improvements.

✓Education and Knowledge Sharing This theme captures 9 of the 84 instances in which developers used ChatGPT to clarify programming concepts, system behaviors, or language-specific features. Instead of providing a direct patch, ChatGPT's explanations helped developers make informed adjustments in pull requests by improving their understanding of technical constraints. A notable example is pull request #2415 (GitHub pull request [2023k](#)) from `digitalbitbox/bitbox-wallet-app` (NE-57), where a developer sought to determine the location of the Homebrew binary installation on a Mac M1 and how different commands influenced the installation paths: *"On a Mac M1, what's the location of the brew binary?"* ChatGPT clarified Homebrew installation differences, helping the developer determine the correct binary path. The pull request discussion validated this approach, with the reviewer finding the insight interesting and raising concerns about system-wide effects, such as *"replace their normal brew [...] all future installs [...] wrong architecture."* While no patch was generated, ChatGPT's response directly influenced the pull request by helping the developer refine installation logic. Understanding how Homebrew installs based on system architecture enabled a more informed decision on script modifications. AI-assisted clarification also guided pull request discussions, leading to further validation and refinement before merging.

✓Debugging and Optimization Strategies We identified 8 of the 84 instances where ChatGPT assisted with debugging, performance optimization, and algorithm refinement, providing strategic insights without generating code snippets. An illustrative example is pull request #5068 (GitHub pull request [2023l](#)) from the `darklang/dark` project (NE-2), where a reviewer sought an alternative to improve the performance of using 'vector' in C++, recalling: *"I remember you mentioning that the performance of pushback isn't great. Do you re-command a different approach?"* The pull request author consulted ChatGPT for performance evaluation and alternative strategies. After reviewing the AI-generated insights, the author proposed a new approach: *"[...] appending it to the front and then reversing should*

do it [...]” The developer switched to using `push` instead of `pushBack`, improving performance since inserting at the front typically has $O(1)$ time complexity, whereas `pushBack` could lead to $O(n)$. This discussion shaped the pull request decision, as ChatGPT provided optimization strategies rather than a direct coding solution. Although no patch was generated, ChatGPT influenced pull request decisions by helping the developer assess trade-offs and select a more efficient approach. AI-assisted insights guided the pull request review, enabling an informed optimization decision before merging.

Summary of RQ3 Results: *The analysis of 84 NE instances reveals that ChatGPT does not always generate patches but still influences pull request decision-making through conceptual guidance, documentation refinement, knowledge sharing, and debugging strategies. Developers used ChatGPT’s responses to improve variable naming, adjust documentation language, refine installation scripts, and explore performance optimizations. These non-patch responses shaped pull request discussions by providing explanations and alternative approaches, demonstrating that AI contributes to software development beyond code generation.*

These results build on and extend prior research on AI-assisted software development by highlighting a distinct usage pattern: developers often consult ChatGPT not for code generation but for conceptual refinement, documentation improvement, and knowledge support. While previous work has emphasized correctness limitations and patch quality issues in AI-generated code (Tanzil et al. 2024; Li et al. 2024), as well as the role of large language models in review automation (Tufano et al. 2024b), our findings point to a complementary function. They align with emerging views of large language models as collaborative assistants that support design thinking, aid code comprehension, and enhance communication, even in the absence of generated patches. These non-code uses suggest that ChatGPT contributes to software development in ways that extend beyond generation, offering new forms of value in pull request workflows that remain underexplored in prior literature.

4.5.1 Implications for RQ3

Our findings highlight that ChatGPT frequently provides non-patch responses that still influence pull request decision-making. Developers do not always require direct patches; instead, they leverage AI-generated feedback for conceptual improvements, documentation refinements, knowledge expansion, and debugging strategies. These insights have important implications for both practitioners and researchers.

Implications for Developers ChatGPT’s non-patch responses help developers refine their code by offering structured feedback on best practices, naming conventions, and maintainability. AI-generated insights can support code reviews, enabling teams to assess design choices and enforce coding standards. In documentation workflows, ChatGPT improves clarity and consistency, allowing teams to incorporate AI-driven language refinements into their review processes. This improves the readability of the documentation and ensures technical accuracy before merging. For debugging and optimization, ChatGPT provides strategic recommendations rather than direct patches, helping developers identify bottlenecks, assess trade-offs, and refine performance strategies. Developers can use these insights to

justify pull request decisions and explore alternative implementations before committing changes. By integrating ChatGPT into pull request workflows, developers can enhance collaborative decision-making, using AI not just for code generation but as a real-time knowledge assistant that strengthens software development practices.

QImplications for Researchers Future research should explore how AI-generated non-patch responses impact long-term software design decisions. Investigating whether repeated exposure to AI-driven conceptual guidance influences coding best practices and architectural choices could provide insights into AI's role in shaping developer decision-making over time. More studies could examine how AI explanations affect debugging efficiency at different experience levels. Research could assess whether AI-driven insights help junior developers bridge knowledge gaps or provide novel perspectives for experienced engineers in complex problem-solving scenarios. Furthermore, analyzing when and why developers trust AI-generated theoretical advice without direct patches could inform the design of AI systems that better align with developer expectations in collaborative software development.

4.6 Results & Discussion – RQ4

How does ChatGPT influence closed pull requests, and what factors contribute to these outcomes under the self-admitted ChatGPT usage?

We identified 53 instances of closed pull requests (CL) for analysis. Following the same sampling methodology used in RQ2 and RQ3, we applied a 95% confidence level with a 5% margin of error, resulting in a stratified sample of 47 CL instances for manual inspection. We then performed an in-depth qualitative analysis of these 47 cases, focusing on the interactions between developers and ChatGPT within the context of the pull request discussions. The qualitative analysis identified six key themes, as illustrated in Fig. 8. The most prevalent reason for pull request closure was attributed to Scope and Quality Issues, which accounted for 31.9% of the cases. Other reasons included procedural and technical adjustments (19.1%), experimental solutions (17.0%), administrative or policy issues (10.6%), duplicate solutions (10.6%) and cases where no specific reason (10.6%) was documented.

✓Scope and Quality Issues This was the leading cause of pull request closures, accounting for 15 of 47 cases. Many pull requests were rejected due to quality concerns, workflow misalignment, or maintainability issues. Although ChatGPT's optimizations were useful, developers struggled to integrate them into long-term architectural plans. In pull request #428 (GitHub pull request 2024e) of the `gemini-hlsw/scheduler` project (CL-31), ChatGPT suggested replacing inefficient `list`-based membership checks with a `set`-based approach and explained Python's `frozenset.issuperset()` behavior along with `__contains__` and `__hash__`. Although this improved efficiency, it did not resolve architectural issues related to object consistency during unpickling. Reviewers referred to ChatGPT's responses to validate assumptions, but ultimately favored a fundamental architectural change over a band-aid fix. They emphasized the need for a long-term solution, stating: *“This needs a proper fix rather than a band-aid solution; future developers will face the same issue if left unresolved [...]”* and *“[...] A generator should be used here. If we proceed with this approach (which raises red flags), it must be carefully revised [...]”*. ChatGPT's optimization was valid, but the main issue was ensuring object consistency dur-

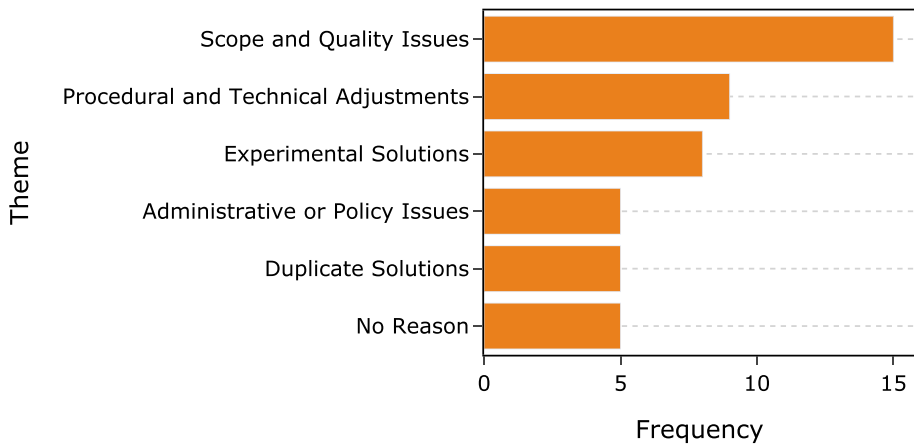


Fig. 8 Showing the distribution of themes for instances where pull requests are closed without merging

ing unpickling. Rather than modifying the membership test, reviewers restructured unpickling by integrating it with `ResourceManager` and relocating it to the `lucupy` library. This case shows that, while ChatGPT-assisted optimizations can enhance performance, they may not align with architectural goals. The reviewers found that its insights were useful for validation, but prioritized structural changes for long-term sustainability.

✓Procedural and Technical Adjustments This theme encompasses pull requests requiring refinements or procedural modifications before integration. 9 of the 47 cases fell into this category, often involving technical restructuring, refactoring, or adherence to project workflows. A representative case is pull request #534 (GitHub pull request [2024f](#)) from `theosanderson/taxonium` - CL-47, which aimed to introduce a map visualization displaying sample origins in a phylogenetic tree. During the discussions, a reviewer explored the implementations using ChatGPT, which suggested leveraging `DeckGL` for rendering pie charts. The reviewer noted: “[...]This was ChatGPT’s idea for that in `DeckGL`[...]”. The pull request author acknowledged the approach’s feasibility, stating: “[...]Neat! I think that approach could work. Let me see if I can get something working in a feature branch [...]”. Although viable, maintainers identified the need for significant back-end modifications, including restructuring metadata handling and creating new API routes. Due to these broader architectural changes, the pull request was closed in favor of continued development within the `add-map-view` feature branch. This case highlights how pull requests requiring substantial refinements are often redirected rather than merged immediately, aligning with trends in this category. ChatGPT played an exploratory role in shaping the implementation strategy, but the final decisions were guided by project-specific requirements.

✓Experimental Solutions 8 of the 47 closed pull requests were later replaced by more refined implementations. These pull requests were often *experimental*, serving as *proof-of-concept* solutions that were ultimately superseded by *optimized alternatives*. In some cases, developers identified existing solutions that made ChatGPT-assisted contributions unnecessary for direct integration. For example, pull requests #2214 (GitHub pull request

2024g) and #2212 (GitHub pull request 2024h) in the `open-learning-exchange/myplanet` repository (CL-29 and CL-36) introduced ChatGPT-generated French and Arabic translations for UI strings, addressing issues #2216⁶ and #2219⁷. However, both pull requests were later closed, with their contributions consolidated into pull request #2229⁸, which was merged. This closure pattern aligns with the common practice of refining and merging multiple contributions into a single comprehensive pull request. Although ChatGPT facilitated the initial translation drafts, final refinements were likely developer-led to ensure linguistic accuracy and adherence to project standards. This underscores how ChatGPT-assisted contributions often act as stepping stones toward more polished, project-aligned solutions rather than directly leading to merged changes.

✓Administrative or Policy Issues five pull requests were closed due to administrative hurdles, including issues of the signing of the Contributor License Agreement (CLA), policy violations and housekeeping rules such as handling stale pull requests. A notable example is pull request #468 (GitHub pull request 2024i) in the `labdao/plex` project – CL-5, where ChatGPT assisted in the development of a well-structured `README.md` for Gnina on the PLEX platform. Despite the quality of the documentation, the pull request was eventually closed after being flagged as stale. This finding highlights that ChatGPT’s assistance does not always lead to pull request integration, as non-technical factors can prevent contributions from being merged, aligning with the goal of RQ to understand the role of AI in pull request closures.

✓Duplicate Solutions pull requests in this category were closed because they duplicated existing efforts in the codebase or overlapped with other pull requests addressing the same issue. This includes cases where developers unknowingly submitted redundant pull requests or discovered an existing solution after submission. We identified five instances that reflect this theme. For example, in pull request #3792 (GitHub pull request 2024j) of the `plausible/analytics` project – CL-20, the developer attempted to optimize how imported views are handled per visit by refining calculations in the `stats/base.ex` module. The primary concern was whether repeated expressions in queries to `ClickHouse` led to redundant computations. ChatGPT was consulted to verify if `ClickHouse` recalculates identical expressions multiple times or reuses results for optimization. Based on the response of ChatGPT, the developer confirmed that `ClickHouse` avoids redundant calculations, improving the efficiency of the query. However, pull request #3792 was later closed in favor of pull request #3830, which provided a cleaner and more efficient solution. The author of pull request #3792 acknowledged the duplication, commenting: “[...] *Closing in favor of #3830.*” and later stating: “[...] *Uff, sorry for this [...]. I had absolutely no intention of redoing your work **emoji** Unexpected coincidence **emoji***”. This case highlights that while ChatGPT helped validate an approach, it did not prevent redundant work, as broader project awareness and coordination among contributors played a more decisive role in the pull request’s closure.

⁶<https://github.com/open-learning-exchange/myplanet/issues/2216>

⁷<https://github.com/open-learning-exchange/myplanet/issues/2219>

⁸<https://github.com/open-learning-exchange/myplanet/pull/2229>

✓No Reason This theme captures 5 instances where the reason for pull requests closure was either not documented or remains ambiguous. Without clear explanations or comments from the project maintainers, the rationale for closing these pull requests is left open to interpretation. For instance, pull request #2213 (GitHub pull request [2024k](#)) proposes the addition of Somali translations for various user interface (UI) strings within the `open-learning-exchange/myplanet` project - CL-9. The pull request aims to address issue #2218 by incorporating localized Somali translations for menu items, error messages, buttons, and other textual elements within the application. The developer requested ChatGPT to translate UI strings from English to Somali. The input consisted of structured XML-like `<string>` elements used in Android applications, where each text label has a corresponding translation. ChatGPT provided translations for a comprehensive set of strings. The developer shared these translations in the pull request, leveraging ChatGPT's ability to convert text into Somali while maintaining cultural and contextual relevance. However, the pull request was closed without merging with no explicit reason documented in the pull request discussion.

Prior studies have examined why pull requests are closed, highlighting factors such as contributor unfamiliarity, low code quality, and project maintenance policies (Tsay et al. [2014](#); Gousios et al. [2016](#); Soares et al. [2015](#)). Our findings complement this work by analyzing closures in the context of self-admitted ChatGPT usage. While some traditional reasons persist (e.g., scope misalignment, procedural issues), we identify new dynamics introduced by LLMs, such as AI-generated proofs-of-concept that are later superseded, or optimizations that, though valid, conflict with project-specific architectural norms. These patterns reveal emerging challenges in human–AI collaboration not addressed in earlier PR closure studies, and the persistent factors we observe align with established findings that PRs are often closed due to low code quality, misalignment with project goals, or procedural requirements such as policy compliance and workflow constraints (Tsay et al. [2014](#); Gousios et al. [2016](#); Soares et al. [2015](#)).

💡 Summary of RQ4 Results: *The analysis identified six primary reasons for the closure of pull requests involving ChatGPT: scope and quality issues (31.9%), procedural and technical adjustments (19.1%), experimental solutions (17.0%), administrative or policy constraints (10.6%), duplicate solutions (10.6%), and undocumented reasons (10.6%). The most common challenge stemmed from unresolved quality concerns and misaligned workflows, limiting the direct integration of ChatGPT-generated suggestions. Procedural and technical constraints frequently required restructuring or compliance with project workflows before acceptance. Additionally, many ChatGPT-assisted contributions served as experimental proofs-of-concept rather than final implementations. Other closures resulted from administrative hurdles, redundant contributions, or a lack of documented rationale. These findings highlight ChatGPT's role in shaping pull request outcomes and the factors that influence its effectiveness in software development.*

4.6.1 Implications for RQ4

Our findings indicate that pull request closures involving ChatGPT-assisted contributions are frequently influenced by project alignment, maintainability, and procedural require-

ments. These insights have important implications for both practitioners and researchers, particularly in understanding how AI influences pull request outcomes and what factors contribute to closure decisions.

QFor Developers While pull request closures occur in both AI-assisted and non-AI-assisted contributions, ChatGPT-generated patches introduce specific challenges due to their contextual misalignment and lack of repository awareness. To improve AI-assisted contributions and minimize pull request closures, developers should focus on the following key areas:

- *Assess maintainability and project alignment:* Many AI-generated patches optimize performance, but do not fit long-term project goals. Developers should critically evaluate whether the suggestions of ChatGPT align with the architectural and maintainability requirements of the software before submission.
- *Ensure compliance with repository policies:* pull requests were often closed due to missing procedural requirements, such as incorrect branch policies or lack of testing. Developers should manually verify that AI-generated patches conform to project guidelines.
- *Check for redundant contributions:* AI-generated solutions sometimes duplicated existing efforts. Developers should review open pull requests and repository history before submitting an AI-assisted patch to prevent unnecessary duplication.
- *Engage with maintainers early:* Some AI-assisted pull requests were closed without detailed feedback, indicating possible misalignment with the project goals. Developers should communicate with maintainers before submission to clarify expectations and increase the likelihood of acceptance.

QFor Researchers Research should focus on addressing the specific challenges observed in AI-assisted pull request closures, particularly in areas where ChatGPT-generated contributions frequently fail. Key directions include:

- *Enhancing AI's repository awareness:* ChatGPT lacks knowledge of project-specific guidelines, leading to pull request rejections due to non-compliance with repository policies. Research should explore techniques for integrating AI with real-time repository metadata to improve adherence to project standards.
- *Improving AI-generated code maintainability:* pull requests were frequently closed due to misalignment with the long-term project architecture. Future research should focus on adaptive AI models that analyze project history and architecture to generate more context-aware patches.

4.7 Implications for Educators

The findings of our study provide valuable insights for educators integrating generative AI tools such as ChatGPT into computer science curricula. By analyzing both merged and closed pull requests, along with varying patch integration levels, we highlight key learning opportunities. The classification into PA, PN, and NE cases offers distinct educational value: PA instances demonstrate AI-assisted coding in real-world, with varying integration percentages illustrating how developers selectively apply AI-generated code. PN cases

encourage students to critically assess AI suggestions, understanding why patches may be rejected due to maintenance concerns or project-specific constraints. NE cases emphasize AI's role beyond code generation, offering insights into conceptual learning, debugging strategies, and design discussions.

Additionally, our analysis of closed pull requests highlights the broader implications of AI-assisted contributions. Many pull requests were closed due to *scope misalignment*, *maintainability concerns*, or *procedural issues*, demonstrating that AI-generated solutions must align with long-term project goals and organizational constraints. Educators can use these examples to teach students the importance of human oversight, team collaboration, and reviewer feedback when working with AI-assisted code. This perspective helps students develop a more comprehensive understanding of software engineering workflows beyond patch integration.

These insights are actively applied in **senior design projects at the University of Nevada Las Vegas (UNLV)**, where students analyze real-world instances of ChatGPT-assisted contributions. Case studies from this research are integrated into the curriculum, allowing students to engage in collaborative software development while exploring the ethical use of AI. ChatGPT is utilized for *code review*, *debugging*, *refinement*, and *documentation*, aligning with industry practices. Beyond senior design projects, these findings inform broader coursework: PN cases help students develop critical evaluation skills in programming courses, while NE cases illustrate how AI supports problem solving and iterative learning in software development. By studying both successful and rejected contributions, students gain a balanced perspective on the strengths and limitations of AI, preparing them for responsible AI-assisted development.

4.8 Overall Findings and Discussion (RQ1 - RQ4)

Our study analyzed 338 pull requests across four interaction categories: Patch Applied (PA), Patch Not Applied (PN), No Patch Generated (NE), and Closed (CL), to understand how ChatGPT influences collaborative software development workflows. Across these categories, ChatGPT-generated suggestions were used for a range of tasks, including bug fixing, small refactorings, configuration changes, documentation updates, and conceptual guidance without direct code adoption. Our analysis focuses on how developers evaluate and integrate such suggestions during pull request review, rather than on semantic correctness or long-term code quality. Accordingly, integration outcomes reflect contextual fit, integration effort, and maintainer trust, and rejection should not be interpreted as an objective assessment of code correctness. In this study, we use the term “impact” to refer specifically to the influence of ChatGPT-generated suggestions on developer and maintainer decision-making during pull request review, rather than to any improvement in code quality, correctness, or performance.

Synthesizing findings across RQ1–RQ4, we identify several cross-cutting patterns that clarify how developers evaluate and adapt ChatGPT-generated suggestions in collaborative workflows:

- 1. AI-generated patches are rarely adopted verbatim.** In PA cases, the median integration rate was 25%. Across 89 validated cases, developers most frequently engaged in *structural integration*, *selective extraction*, or *iterative refinement*, reshaping ChatGPT's code to meet project style, maintainability, and architectural constraints. High-integration cases were uncommon, highlighting the importance of context-aware adaptation.

2. ChatGPT adds value even when patches are not applied. In PN cases, developers used ChatGPT as a conceptual starting point, adapting or rejecting its code based on project constraints and maintainability goals. Suggestions informed architectural decisions, validated alternatives, refined documentation, and improved organization, influencing workflows even without direct code integration.

3. Non-code contributions shape development. In NE cases, ChatGPT influenced debugging strategies, naming conventions, design decisions, documentation quality, and knowledge sharing. These interactions demonstrate value beyond patch generation, supporting conceptual refinement and clearer communication.

4. Pull request closures reflect both traditional and AI-specific barriers. Consistent with prior findings on PR rejections (Tsay et al. 2014; Gousios et al. 2016; Soares et al. 2015), many CL cases were closed due to low code quality, scope misalignment, or policy non-compliance. Others reflected AI-specific patterns, such as proof-of-concept suggestions later superseded or optimizations conflicting with architectural constraints, illustrating new dynamics in human–AI collaboration.

5. Developer judgment remains central. Across categories, developers evaluated ChatGPT's output against project requirements, coding standards, and architectural constraints. Suggestions were treated as one input among many, with final decisions shaped by reviewer feedback and domain expertise. AI-assisted contributions are therefore assessed within established review norms rather than granted special authority.

Viewed in the context of prior research on OSS pull request review (Tsay et al. 2014; Gousios et al. 2016; Soares et al. 2015), these findings suggest that developers evaluate AI-generated code using largely the same criteria applied to human-authored contributions, including architectural fit, maintainability, and scope alignment. However, AI-assisted pull requests introduce additional negotiation around reuse: developers more frequently extract fragments, adapt ideas, or discard generated code while retaining conceptual guidance. Rather than exhibiting blanket distrust toward AI-generated code, maintainers treat it as a provisional artifact whose value depends on contextual integration effort. This extends prior PR review models by shifting emphasis from authorship credibility to adaptability and fit in AI-mediated contributions.

To further contextualize these findings, we performed a lightweight stratified analysis based on repository size, using the number of stars as a proxy (median = 96). Pull requests were divided into smaller and larger groups, and we examined whether integration outcomes differ across these contexts. While the distribution of PA, PN, NE, and CL cases showed moderate variation across groups, the overall integration patterns remained consistent. The median integration rate for PA cases was slightly higher in smaller repositories (22.13) than in larger ones (16.94), but this difference does not indicate a substantive shift in behavior. Qualitative behaviors were also consistent across both groups: in PA, developers engaged in structural integration, selective extraction, and iterative refinement; in PN, ChatGPT was used for adaptation and methodological guidance; in NE, it supported conceptual guidance, documentation, and debugging; and in CL, similar categories of rejection reasons were observed. These results suggest that the observed decision-making behaviors are not specific to a particular class of repositories but reflect broader characteristics of AI-assisted pull request workflows.

Although our dataset is limited to self-admitted ChatGPT usage, these decision-making patterns are not ChatGPT-specific. They reflect how developers evaluate externally gener-

ated code suggestions under uncertainty, an interaction increasingly common across agent-assisted programming tools such as IDE copilots and conversational assistants. In this sense, our findings characterize AI-mediated pull request review behavior more broadly, rather than the effects of a particular model or interface.

Temporal scope and model evolution Our dataset spans a period during which ChatGPT underwent multiple updates. While we do not explicitly track model versions, our analysis focuses on developer and maintainer responses to AI-generated suggestions during pull request review, rather than on the intrinsic quality of a specific model release. Across the DevGPT and extended datasets, we did not observe substantively different integration or rejection behaviors attributable to time period. This suggests that the decision-making patterns identified in this study reflect stable review practices in AI-assisted workflows, even as underlying models evolve.

PatchTrack as a Reusable Research Instrument Beyond its use in this study, PatchTrack enables scalable analysis of how AI-generated code is integrated, adapted, or rejected within pull request workflows. Researchers can apply PatchTrack to examine integration behaviors across different repositories, programming languages, or AI tools, and to study how such behaviors evolve over time. It also supports comparative analyses of AI-assisted development practices across projects or ecosystems. While not intended as a developer-facing tool, PatchTrack may also offer practical insights into how AI-generated suggestions are incorporated within development processes, helping teams better understand integration patterns and review dynamics. In this sense, PatchTrack provides a reusable foundation for investigating AI-assisted software development workflows.

Implications for Practice Practitioners can improve AI-assisted pull request workflows by treating ChatGPT-generated code as a provisional draft rather than a drop-in patch. This entails budgeting review effort for selective extraction, modification, or rejection of AI suggestions and encouraging brief disclosure of AI-assisted contributions during review. Such transparency helps distinguish exploratory assistance from deliberate design decisions and reduces ambiguity in collaborative evaluation.

More broadly, effective AI-assisted workflows reinforce existing review norms such as architectural fit and contextual reasoning rather than bypassing them. For tool builders, this highlights the value of systems that support contextual adaptation and explanation over verbatim generation. For researchers and educators, our findings underscore the importance of studying and teaching not only AI-based code generation but also critical review and adaptation in collaborative development.

5 Related Work

This section positions our study within three strands of research: pull request decision-making, AI-assisted software engineering, and clone detection techniques for patch comparison. Together, these areas provide the foundation for analyzing how ChatGPT-generated suggestions are evaluated and integrated in pull request workflows.

5.1 Pull Request Decision-Making

In collaborative software engineering and open-source projects, the decision-making process for pull requests is crucial to determine both the quality and the efficiency of the development. Gousios et al. (2014) and Yu et al. (2015) have examined how contributions are evaluated, highlighting the importance of peer review and social interactions. Various factors influence pull request acceptance, including technical merit, code quality, and alignment with project goals. For example, Tsay et al. (2014) found that reviewers prioritize maintainability, readability, and adherence to coding guidelines when evaluating contributions. Similarly, Zhao et al. (2019) showed that pull requests that lack comprehensive tests are more likely to be rejected, reinforcing the importance of thorough validation. Complementing these findings, Soares et al. (2015) identified interaction history and test inclusion as strong predictors of pull request acceptance, laying a baseline for comparison with emerging AI-assisted workflows.

Beyond technical factors, social and ecosystem-wide dynamics also influence pull request decision-making. Dey and Mockus (2020) examined how technical attributes and the reputation of contributors affect the acceptance of pull requests in the NPM ecosystem, highlighting the importance of project constraints and community participation. Storey et al. (2017) further emphasized that communication dynamics across different channels profoundly shape participatory cultures in software development, underscoring the collaborative challenges that arise in distributed workflows such as pull requests. In addition to social factors, prior research has highlighted the value of mining developer disclosures to better understand development practices. Yu et al. (2022) introduced *Jitterbug*, a machine learning-based approach for detecting self-admitted technical debt (SATD) in code comments. Inspired by this idea, our study extends the concept of self-admitted artifacts to AI-assisted development by analyzing self-admitted ChatGPT usage in PR discussions.

Meanwhile, Azeem et al. (2020) proposed a classification-based recommendation system to predict pull request actions (accept, respond, or reject), helping integrators make more informed decisions within pull-based workflows. Our study builds on this body of research (Ford et al. 2019; Golzadeh et al. 2019; Legay et al. 2018; Zhang et al. 2022) by extending the focus to AI-assisted pull request workflows, examining how ChatGPT influences pull request outcomes. By analyzing merged and closed pull requests, we provide empirical insights into how developers integrate, modify, or reject AI-generated patches, offering guidance on optimizing generative AI tools for improving pull request quality and accelerating decision-making. Prior work characterizes how OSS developers evaluate human-authored pull requests, but largely assumes a human author. Our study extends this work by examining how established evaluation criteria are applied and adapted in AI-assisted pull request workflows.

5.2 AI-Assisted Software Engineering

Research on AI-assisted software engineering has examined the integration of ChatGPT-generated code in open-source projects, with studies such as Grewal et al. (2024) and Siddiq et al. (2024) providing key insights. Grewal et al. analyzed 3,044 ChatGPT-generated code snippets in GitHub repositories, finding that 54% of AI-generated code is retained in projects with minimal modifications. However, their study primarily measured retention

rates and did not investigate the decision-making process behind the acceptance or rejection of the pull request. Similarly, Siddiq et al. evaluated the quality and security of ChatGPT-generated code, finding that AI-generated patches were rarely merged (only 12%) due to missing documentation, undefined variables, and security vulnerabilities such as improper resource management and hard-coded credentials. Recent work by Ehsani et al. (2025) highlights that gaps in developer prompts can lead to ineffective LLM outputs, further complicating the adoption of AI-generated code in collaborative workflows. Preliminary work by Latendresse et al. (2024) assessed ChatGPT's effectiveness in recommending software libraries, raising concerns about licensing issues and integration reliability; however, their focus remains on library usage rather than collaborative coding workflows. Our study builds on this work by examining why ChatGPT-generated patches are accepted, modified, or rejected in pull request workflows, and how these decisions differ from the evaluation of human-authored code.

Beyond studies on AI-generated code, prior work has investigated how automation influences developer workflows and decision-making. Tufano et al. (2024b) found that automated code review tools, including those powered by LLMs, often ignore the human-in-the-loop context, limiting their reliability and developer trust. Cassee et al. (2020); Ghorbani et al. (2023) showed that automated systems such as Continuous Integration (CI) and GitHub bots alter code review behaviors and are critically evaluated by developers based on trust and perceived utility. Extending this line of inquiry, Abedu et al. (2024); Khatoonabadi et al. (2023) analyzed the adoption of LLM-based chatbots and automation bots in pull request workflows, emphasizing challenges related to trust, accuracy, and context sensitivity. Consistent with these concerns, Tanzil et al. (2024) proposed a technique for detecting incorrect AI-generated code suggestions during code reviews, further underscoring the risks of hallucinated outputs.

In addition to studies on AI-generated code retention and quality, researchers have broadly examined AI-assisted software development, including the roles of ChatGPT and GitHub Copilot in code reviews, debugging, and software documentation (Jiang et al. 2023; Guo et al. 2024; Deng et al. 2024; Hao et al. 2024; Vaithilingam et al. 2022; Liang et al. 2024; Peng et al. 2023). Although these studies demonstrate improvements in productivity and code quality, they often focus on controlled experiments rather than real-world open-source development (Hou et al. 2023; Ju et al. 2023; Siddiq et al. 2023). Recent work by Xiao et al. (2024) analyzed how developers edit and adopt AI-generated PR descriptions, identifying frequent partial reuse and frequent human intervention – patterns echoed in our analysis of ChatGPT-suggested patches. Previous work has explored the role of AI in pull request workflow automation (Champa et al. 2024; Chouchen et al. 2024), but little research has examined how developers interact with ChatGPT in pull request discussions and adapt AI-generated patches to fit project-specific constraints. Our work addresses this gap by empirically analyzing ChatGPT's impact in pull request workflows, particularly the decision-making processes that influence patch acceptance, modification, or rejection. This challenge is further complicated when dealing with multi-hunk patches, which often require semantic coordination and broader context management. Nashid et al. (2025) recently showed that LLMs struggle with such coordination across hunk boundaries, a limitation also observed in our dataset.

Prior research on software evolution and API maintenance has also shown that developers often face challenges when integrating changes due to harmful API usage patterns and

breaking changes. Ochoa et al. (2025) systematically characterized harmful API uses and available repair techniques, while Ochoa et al. (2022) demonstrated how semantic versioning violations and API instability can complicate integration efforts. These challenges parallel the issues we observe when developers critically assess and adapt ChatGPT-suggested patches to fit evolving project architectures and avoid integration risks.

5.3 Code Clone Detection

PatchTrack uses clone detection techniques to determine if a patch was applied, aligning our work with prior studies on clone detection (Min and Li Ping 2019; Zhang and Sakurai 2021; Ramkisoen et al. 2022; Jang et al. 2012; Kim et al. 2017b). Baker (1995) introduced a system for detecting verbatim code copying through text-based clone detection, while Kamiya et al. (2002) developed CCFinder, a token-based tool that converts code into token sequences for similarity comparison—a method also employed in tools like Sourcercc (Sajnani et al. 2016), Rebedug (Jang et al. 2012), PaReco (Ramkisoen et al. 2022), and VUDDY (Kim et al. 2017a). Baxter et al. (1998) demonstrated the use of abstract syntax trees (ASTs) for clone detection by identifying functional similarities despite syntax differences, as further emphasized by Lingxiao et al.’s DECKARD (Jiang et al. 2007). More recent studies leverage graph analysis and machine learning; for instance, Bowman et al.’s VGRAPH (Bowman and Huang 2020) uses code property triplets, while Feng et al. (2024) combine token-based and ML techniques, reporting improved performance over traditional approaches. Beyond general-purpose clone detection, research on framework-specific clone analysis has provided insights into software ecosystems. Businge et al. (2019) and Kawuma et al. (2016) studied clones in the Eclipse framework, reporting that clone occurrences range between 9% and 10%. These findings illustrate the prevalence of duplicated code even in structured development environments, reinforcing the importance of precise clone detection techniques, such as those implemented in PatchTrack, to enhance software quality and maintainability.

6 Threats to Validity and Trustworthiness

6.1 Threats to Validity

For RQ1, the primary threat to validity lies in the accuracy of PatchTrack. Reliance on n -gram tokenization with $n = 1$ may introduce false positives, and normalization steps can obscure structural differences between ChatGPT-generated patches and applied code. Because PatchTrack focuses on exact token overlap rather than semantic equivalence, substantially modified patches may be classified as PN even when ChatGPT influenced the final implementation. To mitigate this threat, we manually validated a subset of cases and refined the classification procedure, though some edge cases remain. Sensitivity analysis using the manually validated subset shows that the relative distribution of PA, PN, and NE classifications and the median integration rate remain consistent with the full dataset, suggesting that the aggregate findings reported in RQ1 are robust to potential classification noise.

Another threat concerns dataset representativeness. Our analysis is limited to pull requests where developers explicitly shared ChatGPT-generated content, excluding private or undisclosed AI usage. This may introduce selection bias, as the dataset captures only visible instances of AI assistance. However, we view self-admitted usage as a conservative attribution strategy that prioritizes validity over coverage, and the observed decision-making patterns are not specific to ChatGPT but reflect how developers evaluate externally generated code suggestions in pull request workflows.

A related threat concerns temporal effects due to model evolution. Our dataset spans multiple ChatGPT updates, which may influence the nature of generated suggestions. However, because our analysis focuses on developer decision-making and integration behavior rather than model performance, and because we did not observe qualitatively different adoption patterns across time periods, we believe this does not threaten the validity of the reported findings.

A further limitation is the absence of a baseline comparison with non-AI-assisted pull requests. Such comparisons are challenging because AI usage is often undisclosed, making it difficult to construct reliable control samples. While historical pre-ChatGPT pull requests could serve as a proxy, temporal confounds such as changes in tooling, practices, and norms complicate interpretation. We therefore focus on self-admitted ChatGPT usage (SACU) and leave broader comparisons to future work.

Finally, a construct validity threat concerns the interpretation of “impact.” As clarified in our discussion, impact in this study refers to developer decision-making and integration outcomes during pull request review, rather than semantic correctness or long-term code quality. Acceptance, adaptation, or rejection of AI-generated suggestions represent the most immediate and observable effects of AI assistance in collaborative development, but they do not imply intrinsic code quality.

6.2 Trustworthiness of Qualitative Findings (RQ2, RQ3, RQ4)

For RQ2, RQ3, and RQ4, we assess the trustworthiness of our qualitative findings using Lincoln and Guba’s framework (Lincoln and Guba 1985).

Credibility *We employed a rigorous qualitative analysis protocol based on the Framework Method and thematic extraction using card sorting (Gale et al. 2013; Spencer 2009). For RQ1, a sample of 230 pull requests (90 PA, 56 PN, and 84 NE) was manually validated to assess the accuracy of PatchTrack’s automated classification. This process was conducted independently by the two authors using shared labeling criteria, achieving a Cohen’s Kappa of 0.85.*

For RQ1–RQ4, we applied a multi-stage qualitative process involving one author, a research assistant, and a senior researcher. Structured summaries were independently created for overlapping subsets, consolidated through collaborative review, and coded into themes using card sorting. Disagreements were resolved through consensus meetings. All findings were grounded in developer conversations and pull request excerpts, with examples and artifacts included in the replication package (Ogenrwot and Businge 2025).

Transferability *Although derived from GitHub pull request discussions, our findings may transfer to other AI-assisted software development workflows. We provide detailed descriptions of the dataset, coding process, and themes to support contextual assessment.*

Dependability *We document all coding procedures, classification criteria, and iterative refinements to ensure consistency and support replication.*

Confirmability *To minimize researcher bias, thematic extraction followed structured decision rules and was cross-verified across independent coding rounds. Direct excerpts ensure interpretations remain grounded in the data.*

6.3 Limitations and Future Work

One limitation of this study is the lack of explicit information about the ChatGPT versions used in the analyzed conversations. As model capabilities evolve, different versions may influence developer engagement and patch integration differently, but such distinctions could not be assessed with the available data.

Additionally, although our dataset spans multiple projects, we did not examine project-level factors such as repository activity or programming language, which may affect AI-assisted patch integration.

Our analysis is also limited to pull requests with self-admitted ChatGPT usage (SACU), excluding undisclosed or private AI interactions. This may introduce selection bias, as the dataset captures only visible AI assistance.

Future work should investigate how AI-generated recommendations can better incorporate project-specific constraints and maintainability considerations. Longitudinal studies examining the downstream evolution of ChatGPT-assisted patches and their effects on software quality and technical debt would complement our focus on developer decision-making and integration outcomes.

7 Ethical Implications

Ethical integrity is paramount in research utilizing public platforms like GitHub. While repositories are publicly accessible, privacy considerations remain crucial as our analysis involves examining pull requests without explicit consent. To address this, we ensured proper attribution by explicitly acknowledging all pull requests that contributed to our study and discussion. This includes citing relevant pull requests in our references and, where applicable, linking directly to their commits to appropriately credit the contributors.

To promote transparency and reproducibility, we have made our replication package publicly available, including datasets, analysis scripts, and supplementary documentation (Ogenrwot and Businge 2025). These measures uphold the ethical responsibilities of using open-source data while fostering reproducibility and openness in computer science research.

8 Conclusion

This study examined how developers engage with ChatGPT in pull request workflows by analyzing 338 real-world instances of self-admitted AI usage. By classifying cases into four categories (patch applied, patch not applied, no patch generated, and closed), we characterize how developers adapt, reject, or conceptually reuse AI-generated suggestions, revealing how AI assistance shapes decision-making in collaborative development.

Our analysis shows that full adoption of ChatGPT-generated patches is uncommon, with a median integration rate of 25%. Qualitative analysis revealed recurring patterns of *structural integration*, *selective extraction*, and *iterative refinement*, indicating that developers typically treat AI output as a starting point rather than a final solution. Even when no code is integrated, ChatGPT influences pull requests through conceptual guidance, documentation revisions, and debugging strategies. In closed pull requests, we observed both traditional barriers (e.g., code quality, scope misalignment) and AI-specific dynamics (e.g., discarded proofs-of-concept), underscoring the central role of developer judgment.

Consistent with our focus on developer decision-making, this study examines the immediate and observable effects of AI assistance during pull request review rather than semantic correctness or long-term code quality. In practice, our findings suggest that teams can more effectively incorporate AI assistance by treating AI-generated code as a provisional artifact, explicitly reviewing and adapting it for contextual and architectural fit rather than expecting direct reuse.

A PatchTrack Design, Validation, and Robustness Analysis

A.1 PatchTrack Design and Implementation

PatchTrack was adapted from PaReco (Ramkisoen et al. 2022) to detect and classify ChatGPT-influenced patches in pull requests. While PaReco was designed for clone detection across divergent variant forks, it is not directly applicable to ChatGPT-assisted workflows. PatchTrack instead identifies ChatGPT-shared links, extracts code snippets from conversations, and compares them against pull request diffs.

Unlike PaReco, which relies on Bloom filters and can incur higher false positive rates, PatchTrack uses temporary hash tables for token matching. Hash tables offer average-case lookup time of $O(1)$ (Cormen et al. 2009; Bloom 1970), enabling precise token-level matching with reduced false positives.

A.2 Normalization and Language Support

To ensure robust comparison across diverse codebases, PatchTrack applies normalization to both ChatGPT snippets and pull request diffs. This includes removing whitespace and comments, converting to lowercase, filtering non-ASCII characters, and applying syntax-aware heuristics based on file type. The tool currently supports 38 file types, covering all major languages in our dataset.

A.3 Patch Classification Procedure

Each pull request is analyzed at the hunk level. For each hunk, PatchTrack compares the n -gram tokens extracted from added lines (prefixed with '+') to tokens in the ChatGPT snippet:

- If at least one match is found, the hunk is labeled PA .
- If no match is found but a ChatGPT snippet exists, the hunk is labeled PN .
- If no ChatGPT snippet exists, hunks are labeled NE .
- Unsupported file types are labeled CC ; processing failures are labeled EE .Pull request-level labels are derived from the aggregation of hunk-level classifications.

A.4 N-Gram Sensitivity Analysis

We evaluated PatchTrack under n -gram sizes $n = 1, 2, 3, 4$. Table 2 reports the distribution of pull requests across PA , PN , and NE categories for each configuration.

Larger n -gram sizes reduced recall and increased PN classifications due to stricter matching requirements, motivating the choice of $n = 1$.

A.5 Manual Validation and Performance Metrics

We randomly sampled 230 pull requests (90 PA , 56 PN , 84 NE) using a 95% confidence level and 5% margin of error. Each case was independently labeled by the two authors using shared criteria. Inter-rater agreement reached a Cohen's Kappa of 0.85.

Table 3 summarizes performance metrics across n -gram configurations.

A.6 Error Analysis and Robustness

Manual inspection of misclassified cases under $n = 1$ revealed seven errors: four false PA and three false PN classifications. Two primary error sources were identified:

- *GitHub API limitations*: Diffs larger than 400 lines are truncated, leading to missed matches.
- *Single-line overlaps*: Short, syntactically generic lines occasionally triggered matches without semantic relevance. To assess sensitivity to such errors, we recomputed the main RQ1 statistics using only the manually validated subset. The relative distribution of PA , PN , and NE classifications and the median integration rate remained consistent with

Table 2 Patch classification outcomes across different n -gram sizes based on 285 pull requests

Classification Type	$n = 1$	$n = 2$	$n = 3$	$n = 4$
Patch Applied (PA)	116 (40.7%)	83 (29.1%)	47 (16.5%)	38 (13.3%)
Patch Not Applied (PN)	63 (22.1%)	96 (33.7%)	132 (46.3%)	141 (49.5%)
No Patch Exists (NE)	106 (37.2%)	106 (37.2%)	106 (37.2%)	106 (37.2%)

Table 3 PatchTrack performance metrics (accuracy, precision, recall, F1) for each n-gram value

N-Gram	Class	Acc.	Prec.	Recall	F1
n = 1	PA	97.2	96.6	95.6	96.1
	PN	95.4	92.9	94.5	93.7
	NE	100	100	100	100
	Overall	97.5	96.5	96.7	96.6
n = 2	PA	95.6	95.3	88.4	91.7
	PN	95.3	90.3	96.2	93.2
	NE	100	100	100	100
	Overall	96.9	95.2	94.7	94.9
n = 3	PA	95.9	88.1	88.1	88.1
	PN	95.9	94.9	94.9	94.9
	NE	100	100	100	100
	Overall	97.2	94.3	94.3	94.3
n = 4	PA	96.7	90.9	85.7	88.2
	PN	94.5	95.2	97.5	96.3
	NE	100	100	100	100
	Overall	97.1	95.3	94.4	94.8

the full dataset, indicating that the reported aggregate trends are robust to classification noise.

B Qualitative Theme Definitions

This appendix provides the operational definitions and coding criteria for the six qualitative themes used to analyze ChatGPT’s contribution in pull request workflows (RQ1). These themes were applied during manual coding of validated PA cases and reflect distinct modes of developer engagement with AI-generated output. Each pull request instance was assigned a single dominant theme based on the primary role ChatGPT played in the developer’s decision-making process.

C Qualitative Themes for RQ2–RQ4

This appendix provides concise operational definitions for the qualitative themes used to analyze pull requests where ChatGPT-generated patches were not applied (PN), no patches were generated (NE), or pull requests were closed (CL). These themes characterize how ChatGPT influenced developer decision-making even when code was not directly integrated (Tables 4, 5, 6 and 7).

Table 4 Operational definitions of qualitative themes used for coding ChatGPT's contribution in pull requests

Theme	Operational Definition
Constraint Handling	ChatGPT generated code or suggestions that required modification due to project-specific constraints, such as API limitations, architectural rules, dependency requirements, or framework conventions. Developers selectively applied or reshaped AI output to comply with these constraints.
Direct Implementation	ChatGPT-generated code was integrated with little or no modification. The suggested patch aligned closely with project requirements and coding standards, enabling near one-to-one reuse.
Iterative Refinement	Developers used ChatGPT's output as an initial draft and incrementally refined, corrected, or extended it through multiple revisions. This often involved reviewer feedback, testing, or additional design adjustments.
Knowledge Support	ChatGPT contributed primarily through conceptual guidance rather than direct code integration. This includes explanations, documentation suggestions, design advice, or exploratory reasoning that informed developer decisions without being directly copied into the codebase.
Selective Extraction	Developers extracted specific fragments or ideas from ChatGPT's output (e.g., a function, logic pattern, or configuration snippet) while discarding the remainder. The AI served as a partial solution source rather than a complete implementation.
Structural Integration	ChatGPT-generated code was adapted to fit within the project's existing structure, APIs, or architectural patterns. This involved non-trivial restructuring to align with the surrounding codebase and system design.

Table 5 Qualitative themes for Patch Not Applied (PN) cases

Theme	Operational Definition
Methodological Guidance	ChatGPT influenced the developer's approach, reasoning, or solution strategy without its suggestions being directly applied as code.
Internal Improvements and Documentation	ChatGPT supported internal refactoring, documentation updates, or project maintenance activities that did not require direct patch reuse.
Specific Functionality Enhancement	ChatGPT informed improvements to specific features or behaviors, but developers implemented customized solutions rather than applying the suggested patch.
Technical Limitations and Preferences	AI-generated patches were rejected due to technical constraints, architectural incompatibilities, or a preference for alternative implementations.
Adaptation and Tailored Solutions	Developers adapted ChatGPT's conceptual advice to fit project-specific requirements, resulting in customized solutions rather than direct reuse.
Clarification and Correction	ChatGPT helped clarify misunderstandings or correct reasoning, leading developers to revise their approach instead of applying new code.

Table 6 Qualitative themes for No Patch Generated (NE) cases

Theme	Operational Definition
Conceptual Guidance and Theoretical Advice	ChatGPT provided high-level guidance on design principles, programming concepts, optimization strategies, or naming practices without producing concrete code.
Documentation, Communication, and Translation	ChatGPT assisted with documentation clarity, communication refinement, localization, or translation to improve accessibility and understanding.
Debugging and Optimization Strategies	ChatGPT suggested debugging techniques, performance optimizations, or problem-solving strategies without supplying specific code patches.
Education and Knowledge Sharing	ChatGPT was used to explain language features, code behavior, or concepts for learning and understanding rather than implementation.

Table 7 Qualitative themes for Closed pull requests (CL)

Theme	Operational Definition
Administrative or Policy Issues	Pull requests closed due to administrative constraints, policy violations, contributor agreement issues, or repository housekeeping rules.
Experimental Solutions	Proposed changes were exploratory or superseded by alternative implementations, making the pull request unnecessary.
Duplicate Solutions	The pull request duplicated existing functionality or overlapped with another contribution addressing the same issue.
No Reason Stated	The pull request was closed without an explicit explanation or with insufficient context to infer the reason.
Procedural and Technical Adjustments	Closure resulted from unmet procedural requirements or unresolved technical refinements needed for acceptance.
Scope and Quality Issues	Pull requests were closed due to excessive scope, quality concerns, workflow misalignment, or maintainability challenges.

Author Contributions John Businge: Conceptualization, supervision, and review and editing of the manuscript. Daniel Ogenrwot: Data collection, analysis, and writing of the original draft.

Funding This research was supported by the NEVADA NASA EPSCoR program under Award No. AWD2536/GR20392 and NSF Grant Award No. #2519136.

Data Availability The datasets, analysis scripts, and documentation are publicly available at <https://zenodo.org/records/14978625>

Declarations

Ethics approval Not applicable

Informed consent Not applicable

Conflict of interest The authors declare that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Clinical trial number in the manuscript Not applicable

References

- Abedu S, Abdellatif A, Shihab E (2024) Llm-based chatbots for mining software repositories: Challenges and opportunities. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering. EASE '24, pp 201–210. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3661167.3661218>
- Azeem MI, Panichella S, Di Sorbo A, Serebrenik A, Wang Q (2020) Action-based recommendation in pull-request development. In: Proceedings of the International Conference on Software and System Processes. ICSSP '20, pp 115–124. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3379177.3388904>
- Bacchelli A, Bird C (2013) Expectations, outcomes, and challenges of modern code review. In: 2013 35th International Conference on Software Engineering (ICSE), pp 712–721. <https://doi.org/10.1109/ICSE.2013.6606617>
- Baker BS (1995) On finding duplication and near-duplication in large software systems. In: Proceedings of 2nd working conference on reverse engineering, pp 86–95. IEEE
- Baxter JD, Yahin A, Moura L, Sant'Anna M, Bier L (1998) Clone detection using abstract syntax trees. In: Proceedings international conference on software maintenance (Cat. No. 98CB36272), pp 368–377. IEEE
- Bloom BH (1970) Space/time trade-offs in hash coding with allowable errors. *Commun ACM* 13(7):422–426. <https://doi.org/10.1145/362686.362692>
- Bowman B, Huang HH (2020) Vgraph: A robust vulnerable code clone detection system using code property triplets. In: 2020 IEEE European Symposium on Security and Privacy (EuroS&P), pp 53–69. <https://doi.org/10.1109/EuroSP48549.2020.00012>
- Businge J, Kawuma S, Openja M, Bainomugisha E, Serebrenik A (2019) How stable are eclipse application framework internal interfaces? In: 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER), pp 117–127. <https://doi.org/10.1109/SANER.2019.8668018>
- Businge J, Decan A, Zerouali A, Mens T, Demeyer S, De Roover C (2022) Variant forks – motivations and impediments. In: Proceedings of the 29th edition of the IEEE international conference on software analysis, evolution and reengineering, pp 867–877. IEEE Computer Society. <https://doi.org/10.1109/SANER53432.2022.00105>
- Cassee N, Vasilescu B, Serebrenik A (200) The Silent Helper: The Impact of Continuous Integration on Code Reviews . In: 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER), pp 423–434. IEEE Computer Society, Los Alamitos, CA, USA. <https://doi.org/10.1109/SANER48275.2020.9054818>, <https://doi.ieeecomputersociety.org/10.1109/SANER48275.2020.9054818>
- Champa AI, Rabbi MF, Nachuma C, Zibran MF (2024) Chatgpt in action: Analyzing its use in software development. In: Proceedings of the 21st International Conference on Mining Software Repositories. MSR '24, pp 182–186. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3643991.3645077>
- Chen D, Liu Y, Zhou M, Zhao Y, Wang S, Wang X, Chen X, Bissyandé T, Klein J (2024) Llm for mobile: An initial roadmap. *ACM Trans Softw Eng Methodol* 33(5):1–44
- Chouchen M, Bessghaier N, Begoug M, Ouni A, Alomar E, Mkaouer MW (2024) How do software developers use chatgpt? an exploratory study on github pull requests. In: Proceedings of the 21st International Conference on Mining Software Repositories. MSR '24, pp 212–216. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3643991.3645084>
- Cormen TH, Leiserson CE, Rivest RL, Stein C (2009) Introduction to Algorithms, 3rd edn. The MIT Press
- Deng Y, Xia CS, Yang C, Zhang SD, Yang S, Zhang L (2024) Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24, pp 1–13. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3597503.3623343>
- Dey T, Mockus A (2020) Effect of technical and social factors on pull request quality for the npm ecosystem. In: Proceedings of the 14th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM). ESEM '20. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3382494.3410685>

- Ebert C, Louridas P (2023) Generative ai for software practitioners. *IEEE Softw* 40(4):30–38. <https://doi.org/10.1109/MS.2023.3265877>
- Ehsani R, Pathak S, Chatterjee P (2025) Towards detecting prompt knowledge gaps for improved llm-guided issue resolution. In: *Proceedings of the 22nd International Conference on Mining Software Repositories (MSR 2025)*. ACM, Ottawa, Canada. To appear
- Feng S, Suo W, Wu Y, Zou D, Liu Y, Jin H (2024) Machine learning is all you need: A simple token-based approach for effective code clone detection. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24*, pp 1–13. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3597503.3639114>
- Ford D, Behroozi M, Serebrenik A, Parnin C (2019) Beyond the code itself: how programmers really look at pull requests. In: *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Society. ICSE-SEIS '19*, pp 51–60. IEEE Press. <https://doi.org/10.1109/ICSE-SEIS.2019.17>
- Gale NK, Heath G, Cameron E, Rashid S, Redwood S (2013) Using the framework method for the analysis of qualitative data in multi-disciplinary health research. *BMC Med Res Methodol* 13(1):117. <https://doi.org/10.1186/1471-2288-13-117>
- Ghorbani A, Cassee N, Robinson D, Alami A, Ernst NA, Serebrenik A, Wasowski A (2023) Autonomy is an acquired taste: Exploring developer preferences for github bots. In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp 1405–1417. <https://doi.org/10.1109/ICSE48619.2023.00123>
- GitHub pull request (2023). <https://github.com/darklang/dark/pull/5068>
- GitHub pull request (2023a). <https://chatgpt.com/share/8cb16814-2855-4fbd-87e5-bde8ba349728>
- GitHub pull request (2023b). <https://github.com/faker-js/faker/pull/2405>
- GitHub pull request (2023c). <https://github.com/laravel-json-api/core/pull/12>
- GitHub pull request (2023d). <https://github.com/ory/elements/pull/171>
- GitHub pull request (2023e). <https://github.com/darklang/dark/pull/5063>
- GitHub pull request (2023f). <https://github.com/sveltejs/learn.svelte.dev/pull/522>
- GitHub pull request (2023g). <https://github.com/darklang/dark/pull/5058>
- GitHub pull request (2023h). <https://github.com/Bananapus/nana-core/pull/37>
- GitHub pull request (2023i). <https://github.com/codecrafters-io/frontend/pull/1061>
- GitHub pull request (2023j). <https://github.com/faker-js/faker/pull/2230>
- GitHub pull request (2023k). <https://github.com/digitalbitbox/bitbox-wallet-app/pull/2415>
- GitHub pull request (2024a). <https://github.com/pokt-network/poktrol/pull/185>
- GitHub pull request (2024b). <https://github.com/Mudlet/Mudlet/pull/7123>
- GitHub pull request (2024c). <https://github.com/nylas/nylas-python/pull/279>
- GitHub pull request (2024d). <https://github.com/alshedivat/al-folio/pull/2059>
- GitHub pull request (2024e). <https://github.com/gemini-hlsw/scheduler/pull/428>
- GitHub pull request (2024f). <https://github.com/theosanderson/taxonium/pull/534>
- GitHub pull request (2024g). <https://github.com/open-learning-exchange/myplanet/pull/2214>
- GitHub pull request (2024h). <https://github.com/open-learning-exchange/myplanet/pull/2212>
- GitHub pull request (2024i). <https://github.com/labdao/plex/pull/468>
- GitHub pull request (2024j). <https://github.com/plausible/analytics/pull/3792>
- GitHub pull request (2024k). <https://github.com/open-learning-exchange/myplanet/pull/2213>
- GitHub: GitHub REST API Documentation (2025) GitHub. <https://docs.github.com/en/rest?apiVersion=2022-11-28>
- GitHub: Online Appendix (2025) GitHub. https://www.gnu.org/software/diffutils/manual/html_node/Hunk_s.html
- GitHub: The State of Open Source: Octoverse 2024 (2024). <https://github.blog/news-insights/octoverse/octoverse-2024/>
- Golzadeh M, Decan A, Mens T (2019) On the effect of discussions on pull request decisions. In: *BENEVOLENT Gousios G, Pinzger M, Deursen Av (2014) An exploratory study of the pull-based software development model. In: Proceedings of the 36th International Conference on Software Engineering. ICSE 2014*, pp 345–355. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2568225.2568260>
- Gousios G, Storey M-A, Bacchelli A (2016) Work practices and challenges in pull-based development: the contributor's perspective. In: *Proceedings of the 38th International Conference on Software Engineering. ICSE '16*, pp 285–296. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2884781.2884826>
- Grewal B, Lu W, Nadi S, Bezemer C-P (2024) Analyzing developer use of chatgpt generated code in open source github projects. In: *Proceedings of the 21st International Conference on Mining Software Repositories. MSR '24*, pp 157–161. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3643991.3645072>

- Guo Q, Cao J, Xie X, Liu S, Li X, Chen B, Peng X (2024) Exploring the potential of chatgpt in automated code refinement: An empirical study. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24*, pp 1–13. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3597503.3623306>
- Hao H, Hasan KA, Qin H, Macedo M, Tian Y, Ding SHH, Hassan AE (2024) An empirical study on developers' shared conversations with chatgpt in github pull requests and issues. *arXiv preprint arXiv:2403.10468*. License: CC BY-NC-SA 4.0
- Hassan AE, Lin D, Rajbahadur GK, Gallaba K, Cogo FR, Chen B, Zhang H, Thangarajah K, Oliva GA, Lin J, Abdullah WM, Jiang ZM (2024) Rethinking software engineering in the foundation model era: A curated catalogue of challenges in the development of trustworthy firmware. *arXiv preprint arXiv:2402.15943*
- Hou X, Zhao Y, Liu Y, Yang Z, Wang K, Li L, Luo X, Lo D, Grundy J, Wang H (2023) Large language models for software engineering: A systematic literature review. *arXiv preprint arXiv:2308.10620*
- Howard J, Ruder S (2018) Universal language model fine-tuning for text classification. In: Gurevych I, Miyao Y (eds) *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp 328–339. Association for Computational Linguistics, Melbourne, Australia. <https://doi.org/10.18653/v1/P18-1031>, <https://aclanthology.org/P18-1031>
- Huang Y, Chen Y, Chen X, Chen J, Peng R, Tang Z, Huang J, Xu F, Zheng Z (2024) Generative software engineering. *arXiv preprint arXiv:2403.02583*, <https://doi.org/10.48550/arXiv.2403.02583>. Submitted on 5 Mar 2024, last revised 3 Apr 2024 (this version, v2)
- Jang J, Agrawal A, Brumley D (2012) Redebug: Finding unpatched code clones in entire os distributions. In: *2012 IEEE symposium on security and privacy*, pp 48–62. <https://doi.org/10.1109/SP.2012.13>
- Jiang L, Misserghhi G, Su Z, Glondu S (2007) Deckard: Scalable and accurate tree-based detection of code clones. In: *29th International Conference on Software Engineering (ICSE'07)*, pp 96–105. <https://doi.org/10.1109/ICSE.2007.30>
- Jiang N, Liu K, Lutellier T, Tan L (2023) Impact of code language models on automated program repair. In: *Proceedings of the 45th International Conference on Software Engineering. ICSE '23*, pp 1430–1442. IEEE Press. <https://doi.org/10.1109/ICSE48619.2023.00125>
- Jin K, Wang C-Y, Pham HV, Hemmati H (2024) Can chatgpt support developers? an empirical evaluation of large language models for code generation. In: *Proceedings of the 21st International Conference on Mining Software Repositories. MSR '24*, pp 167–171. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3643991.3645074>
- Ju J, Yu L, Li X, Yang L, Zuo C (2023) Llama-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning. In: *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, pp 647–658. IEEE
- Kamiya T, Kusumoto S, Inoue K (2002) Ccfinder: A multilingual token-based code clone detection system for large scale source code. *IEEE Trans Software Eng* 28(7):654–670
- Kawuma S, Businge J, Bainomugisha E (2016) Can we find stable alternatives for unstable eclipse interfaces? In: *2016 IEEE 24th International Conference on Program Comprehension (ICPC)*, pp 1–10. <https://doi.org/10.1109/ICPC.2016.7503716>
- Khatoonabadi S, Costa DE, Mujahid S, Shihab E (2023) Understanding the helpfulness of stale bot for pull-based development: An empirical study of 20 large open-source projects. *ACM Trans Softw Eng Methodol* 33(2). <https://doi.org/10.1145/3624739>
- Kim S, Woo S, Lee H, Oh H (2017a) Vuddy: A scalable approach for vulnerable code clone discovery. In: *2017 IEEE Symposium on Security and Privacy (SP)*, pp 595–614. <https://doi.org/10.1109/SP.2017.62>
- Kim S, Woo S, Lee H, Oh H (2017b) Vuddy: A scalable approach for vulnerable code clone discovery. In: *2017 IEEE Symposium on Security and Privacy (SP)*, pp 595–614. <https://doi.org/10.1109/SP.2017.62>
- Latendresse J, Khatoonabadi S, Abdellatif A, Shihab E (2024) Is ChatGPT a Good Software Librarian? An Exploratory Study on the Use of ChatGPT for Software Library Recommendations. <https://doi.org/10.48550/arXiv.2408.05128>
- Legay D, Decan A, Mens T (2018) On the impact of pull request decisions on future contributions. *arXiv preprint arXiv:1812.06269*
- Li S, Cheng Y, Chen J, Xuan J, He S, Shang W (2024) Assessing the performance of ai-generated code: A case study on github copilot. In: *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*, pp 216–227. <https://doi.org/10.1109/ISSRE62328.2024.00030>
- Liang JT, Yang C, Myers BA (2024) A large-scale survey on the usability of ai programming assistants: Successes and challenges. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24*, pp 1–13. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3597503.3608128>
- Lincoln YS, Guba EG (1985) *Naturalistic Inquiry*, pp 393–408. Sage Publications, Beverly Hills, Calif. <http://s://archive.org/details/naturalisticinqu00linc>

- Luo L, Ning J, Zhao Y, Wang Z, Ding Z, Chen P, Fu W, Han Q, Xu G, Qiu Y et al (2024) Taiyi: a bilingual fine-tuned large language model for diverse biomedical tasks. *J Am Med Inf Assoc*, 037
- Menzies T, Pecheur C (2020) Software engineering with ai/ml: State of the art and future prospects. *Autom Softw Eng* 27(4):459–489. <https://doi.org/10.1007/s10515-020-00280-2>
- Min H, Li Ping Z (2019) Survey on software clone detection research. In: Proceedings of the 2019 3rd International Conference on Management Engineering, Software Engineering and Service Sciences. ICMSS 2019, pp 9–16. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/312662.3312707>
- Moumoula M, Kabore A, Klein J, Bissyandé T (2024) Cross-lingual code clone detection: When llms fall short against embedding-based classifier. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)
- Nashid N, Ding D, Gallaba K, Hassan AE, Mesbah A (2025) Characterizing multi-hunk patches: Divergence, proximity, and llm repair challenges. arXiv preprint [arXiv:2506.04418](https://arxiv.org/abs/2506.04418)
- Ochoa L, Degueule T, Falleri J-R, Vinju J (2022) Breaking bad? semantic versioning and impact of breaking changes in maven central: An external and differentiated replication study. *Empir Softw Eng* 27(3):61
- Ochoa L, Hammad M, Giray G, Babur O, Bennis K (2025) Characterising harmful api uses and repair techniques: Insights from a systematic review. *Comput Sci Rev* 57:100732
- Ogenrwot D, Businge J (2025) Replication Package for PatchTrack: A Comprehensive Analysis of ChatGPT's Influence on Pull Request Outcomes. <https://doi.org/10.5281/zenodo.14978624>
- OpenAI (2025) Terms of Use. OpenAI. <https://openai.com/policies/terms-of-use>
- Ouédraogo W, Kabore K, Tian H, Song Y, Koyuncu A, Klein J, Lo D, Bissyandé T (2024) Llms and prompting for unit test generation: A large-scale evaluation. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)
- Ozpolat Z, Yildirim Karabatak M (2023) Artificial intelligence-based tools in software development processes: Application of chatgpt. *Eur J Technic*. <https://doi.org/10.36222/ejt.1330631>
- Peng S, Kalliamvakou E, Cihon P, Demirer M (2023) The impact of ai on developer productivity: Evidence from github copilot. arXiv preprint [arXiv:2302.06590](https://arxiv.org/abs/2302.06590), <https://doi.org/10.48550/arXiv.2302.06590>. Submitted on 13 Feb 2023
- Ramkisoen PK, Businge J, Bladel B, Decan A, Demeyer S, De Roover C, Khomh F (2022) Pareco: patched clones and missed patches among the divergent variants of a software family. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ESEC/FSE 2022, pp 646–658. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3540250.3549112>
- Rich C, Waters RC (1990) *The Programmer's Apprentice*. ACM Press/Addison-Wesley, Reading, MA
- Rigby PC, Bird C (2013) Convergent contemporary software peer review practices. In: Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering. ESEC/FSE 2013, pp 202–212. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2491411.2491444>
- Russo D (2024) Navigating the complexity of generative ai adoption in software engineering. *ACM Trans Softw Eng Methodol*. <https://doi.org/10.1145/3652154>. Just Accepted
- Sajjani H, Saini V, Svajlenko J, Roy CK, Lopes CV (2016) Sourcerercc: scaling code clone detection to big-code. In: Proceedings of the 38th International Conference on Software Engineering. ICSE '16, pp 1157–1168. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2884781.2884877>
- Sauvola J, Tarkoma S, Klemettinen M, Riekkä J, Doermann D (2024) Future of software development with generative ai. *Automated Softw Eng* 31(26). <https://doi.org/10.1007/s10515-024-00330-1>
- Siddiq ML, Santos J, Tanvir RH, Ulfat N, Rifat FA, Lopes VC (2023) Exploring the effectiveness of large language models in generating unit tests. arXiv preprint [arXiv:2305.00418](https://arxiv.org/abs/2305.00418)
- Siddiq ML, Roney L, Zhang J, Santos JCDS (2024) Quality assessment of chatgpt generated code and their use by developers. In: Proceedings of the 21st International Conference on Mining Software Repositories. MSR '24, pp 152–156. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3643991.3645071>
- Soares DM, Lima Júnior ML, Murta L, Plastino A (2015) Acceptance factors of pull requests in open-source projects. In: Proceedings of the 30th Annual ACM Symposium on Applied Computing. SAC '15, pp 1541–1546. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2695664.2695856>
- Spencer D (2009) *Card Sorting: Designing Usable Categories*. Rosenfeld Media, New York
- Storey M-A, Zagalsky A, Filho FF, Singer L, German DM (2017) How social and communication channels shape and challenge a participatory culture in software development. *IEEE Trans Softw Eng* 43(2):185–204. <https://doi.org/10.1109/TSE.2016.2584053>

- Tanzil MH, Khan JY, Uddin G (2024) Chatgpt incorrectness detection in software reviews. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3597503.3639194>
- Tsay J, Dabbish L, Herbsleb J (2014) Influence of social and technical factors for evaluating contribution in github. In: Proceedings of the 36th International Conference on Software Engineering. ICSE 2014, pp 356–366. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2568225.2568315>
- Tufano R, Mastropaolo A, Pepe F, Dabić O, Di Penta M, Bavota G (2024a) Unveiling chatgpt's usage in open source projects: A mining-based study. arXiv preprint [arXiv:2402.13456](https://arxiv.org/abs/2402.13456). Paper accepted for publication at 21st International Conference on Mining Software Repositories (MASR'24)
- Tufano R, Dabić O, Mastropaolo A, Ciniselli M, Bavota G (2024b) Code review automation: Strengths and weaknesses of the state of the art. *IEEE Trans Softw Eng* 50(2):338–353. <https://doi.org/10.1109/TSE.2023.3348172>
- Vaithilingam P, Zhang T, Glassman EL (2022) Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In: Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems. CHI EA '22, pp 1–7. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3491101.3519665>
- Wang L, Zheng Z, Wu X, Sang B, Zhang J, Tao X (2023) Fork entropy: Assessing the diversity of open source software projects' forks. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp 204–216. <https://doi.org/10.1109/ASE56229.2023.00168>
- Weeraddana NR, Xu X, Alfadel M et al (2023) An empirical comparison of ethnic and gender diversity of devops and non-devops contributions to open-source projects. *Empirical Softw Eng* 28:150 <https://doi.org/10.1007/s10664-023-10394-9>. Accepted 11 Sept 2023
- Wermelinger M (2023) Using github copilot to solve simple programming problems. In: Proceedings of the 54th ACM technical symposium on computer science education, p 7. ACM, Toronto, Canada. <https://doi.org/10.1145/3545945.3569830>
- White J, Fu Q, Hays S, Sandborn M, Olea C, Gilbert H, Elnashar A, Spencer-Smith J, Schmidt DC (2023) A prompt pattern catalog to enhance prompt engineering with chatgpt. arXiv preprint [arXiv:2302.11382](https://arxiv.org/abs/2302.11382)
- Xiao T, Treude C, Hata H, Matsumoto K (2023) Devgpt: Studying developer-chatgpt conversations. Zenodo. <https://doi.org/10.5281/zenodo.8304091>
- Xiao T, Hata H, Treude C, Matsumoto K (2024) Generative ai for pull request descriptions: Adoption, impact, and developer interventions. *Proc ACM Softw Eng* 1(FSE). <https://doi.org/10.1145/3643773>
- Yu Y, Wang H, Filkov V, Devanbu P, Vasilescu B (2015) Wait for it: determinants of pull request evaluation latency on github. In: Proceedings of the 12th Working Conference on Mining Software Repositories. MSR '15, pp 367–371. IEEE Press
- Yu Z, Fahid FM, Tu H, Menzies T (2022) Identifying self-admitted technical debts with jitterbug: A two-step approach. *IEEE Trans Softw Eng* 48(5):1676–1691. <https://doi.org/10.1109/TSE.2020.3031401>
- Zhang H, Sakurai K (2021) A survey of software clone detection from security perspective. *IEEE Access* 9:48157–48173. <https://doi.org/10.1109/ACCESS.2021.3065872>
- Zhang X, Yu Y, Gousios G, Rastogi A (2022) Pull request decisions explained: An empirical overview. *IEEE Trans Software Eng* 49(2):849–871
- Zhao G, Costa DA, Zou Y (2019) Improving the pull requests review process using learning-to-rank algorithms. *Empir Softw Eng* 24(4):2140–2170. <https://doi.org/10.1007/s10664-019-09696-8>
- Zhu J, Zhou M, Mockus A (2016) Effectiveness of code contribution: from patch-based to pull-request-based tools. In: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering. FSE 2016, pp 871–882. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2950290.2950364>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Daniel Ogenrwot¹  · John Businge¹ 

✉ John Businge
john.businge@unlv.edu

Daniel Ogenrwot
ogenrwot@unlv.nevada.edu

¹ Department of Computer Science, University of Nevada Las Vegas, 4505 S. Maryland Pkwy., Las Vegas 89154, Nevada, United States of America